密级：_____

南昌大学科学技术学院
SCIENCE AND TECHNOLOGY COLLEGE OF NANCHANG UNIVERSITY

学 士 学 位 论 文

THESIS OF BACHELOR

（2023—2027 年）

题　　目　基于知识图谱增强大模型的产线故障诊断系统

学 科 部：信息学科部　系：计算机系

专　　业：软件工程　班级：软件工程 234 班

学生姓名：王禹博　学号：7023123210

指导教师：邓伦丹　职称：【职称】

起讫日期：2026 年 11 月—2027 年 4 月

南昌大学 科学技术学院

学士学位论文原创性申明

本人郑重申明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式表明。本人完全意识到本申明的法律后果由本人承担。

作者签名： **日期：**

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南昌大学科学技术学院可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于

保　密 □，在　　年解密后适用本授权书。

不保密 □。

（请在以上相应方框内打“√”）

作者签名： **日期：**

导师签名：　　日期：

基于知识图谱增强大模型的产线故障诊断系统

摘要

设备故障排查依赖维修人员的个人经验，而企业积累的大量维修记录以口语化短文本形式分散在表格中，难以检索和复用；通用大语言模型缺少企业设备知识，直接用于诊断容易产生与事实不符的内容。针对这些问题，本文以某汽车零部件企业机加工车间 2023—2025 年的 15319 条维修记录为基础，设计并实现了一个基于知识图谱增强大模型的产线故障诊断系统。

首先，设计了包含区域、设备、部件、故障现象、原因和处理措施六类实体的故障本体，采用大模型抽取、规则校验、专家复核与实体规范化相结合的方式，由 2023 年的维修记录构建了包含 6993 个实体节点的故障知识图谱，每条关系保留来源记录编号以便溯源。其次，提出了统一检索方法，基于实体词典识别问句中的设备，以二元字组字面覆盖度与 BGE-M3 向量相似度加权打分，一次检索得到的结果同时用于知识图谱子图、相似记录和决策树。最后，将知识图谱链路与相似记录组织成证据，由大模型生成按原因分支、先易后难的排查决策树，大模型不可用时自动改用规则生成。

实验表明：与精确匹配相比，图谱子图检索的空结果率由 50.3%降至 7.3%；相似记录检索的 Hit@5 达到 95.8%；在 100 条留一法测试问句上，无检索证据时大模型的根因命中率仅为 2%，同时使用知识图谱和相似记录时达到 46%，决策树质量评分（5 分制）平均 3.86 分，显著高于规则方法和无证据的大模型；人工抽检显示六元组整条抽取准确率为 32.7%，原因字段是主要短板；在笔记本 CPU 上检索阶段约 2 秒完成，无网络时整个诊断约 2 秒给出结果。系统部署在企业内网，为维修人员快速了解同类故障的历史原因和处理方法提供了支持。

关键词：知识图谱；大语言模型；检索增强生成；故障诊断；维修记录

Production Line Fault Diagnosis System Based on Knowledge Graph Enhanced Large Language Model

Abstract

Equipment troubleshooting on production lines relies heavily on the personal experience of maintenance staff. Although enterprises have accumulated large numbers of maintenance records, these records are short, colloquial texts scattered across spreadsheets and are hard to search and reuse, while general-purpose large language models (LLMs) lack enterprise-specific equipment knowledge and tend to produce content that is inconsistent with the facts. Based on 15,319 maintenance records collected from 2023 to 2025 in the machining workshop of an automotive parts manufacturer, this thesis designs and implements a production-line fault diagnosis system based on a knowledge-graph-enhanced LLM.

First, a fault ontology with six entity types — area, equipment, component, symptom, cause and action — is designed. By combining LLM-based extraction, rule-based validation, expert review and entity normalization, a fault knowledge graph with 6,993 entity nodes is built from the 2023 records, and every relation keeps the identifier of its source record for traceability. Second, a unified retrieval method is proposed: equipment mentioned in the query is recognized with an entity dictionary, and candidate records are ranked by a weighted combination of IDF-weighted character-bigram coverage and BGE-M3 vector similarity. The results of a single retrieval are shared by the knowledge subgraph, the similar-record list and the decision tree. Finally, the retrieved graph chains and similar records are organized as evidence, from which the LLM generates a troubleshooting decision tree branched by cause and ordered from low to high cost; when the LLM is unavailable, the system automatically falls back to a rule-based decision tree.

Experiments show that, compared with exact matching, the empty-result rate of subgraph retrieval drops from 50.3% to 7.3%; similar-record retrieval reaches a Hit@5 of 95.8%; on 100 leave-one-out test queries, the root-cause hit rate of the LLM is only 2% without retrieved evidence and rises to 46% when both the knowledge graph and similar records are used, and the generated decision trees receive an average quality score of 3.86 out of 5, significantly higher than the rule-based method and the LLM without evidence; a manual check shows that only 32.7% of the extracted six-tuples are fully correct, with the cause field being the main weakness; on a laptop CPU, retrieval takes about two seconds, and the whole diagnosis takes about

two seconds when no network is available. Deployed in the enterprise intranet, the system helps maintenance staff quickly learn the historical causes and solutions of similar faults.

Keywords: knowledge graph; large language model; retrieval-augmented generation; fault diagnosis; maintenance records

目录

THESIS OF BACHELOR I
摘要 I
Abstract II
第 1 章 绪论 1
1.1 研究背景与意义 1
1.2 国内外研究现状 2
1.2.1 设备故障知识图谱 2
1.2.2 维修文本知识抽取 2
1.2.3 检索增强生成与知识图谱增强大模型 2
1.2.4 大模型在工业故障诊断中的应用 3
1.2.5 存在的问题 3
1.3 本文主要工作 3
1.4 论文组织结构 4
第 2 章 相关技术 5
2.1 知识图谱 5
2.1.1 知识图谱的基本概念 5
2.1.2 知识抽取 5
2.1.3 图数据库 Neo4j 6
2.2 文本向量表示与检索 6
2.2.1 基于词项的字面检索 6
2.2.2 稠密向量检索 6
2.2.3 向量数据库 Qdrant 7
2.3 检索结果融合与重排序 7
2.3.1 加权融合与倒数排名融合 7
2.3.2 交叉编码器重排序 7
2.4 大语言模型与检索增强生成 8
2.4.1 大语言模型 8
2.4.2 提示词工程与结构化输出 8
2.4.3 检索增强生成 8
2.4.4 知识图谱增强的大模型 9
2.5 系统开发技术 9
2.5.1 后端：FastAPI 与服务端推送 9
2.5.2 前端：React 与可视化组件 9
2.6 本章小结 10
第 3 章 需求分析 11
3.1 业务现状与问题 11

3.2 业务流程分析 11
3.3 用户角色 12
3.4 功能需求 13
3.4.1 故障诊断 13
3.4.2 知识管理 14
3.4.3 系统配置 14
3.5 非功能需求 14
3.6 可行性分析 15
3.7 本章小结 15
第 4 章 总体设计 16
4.1 设计原则 16
4.2 系统架构 16
4.3 功能模块设计 17
4.4 故障知识图谱本体设计 18
4.4.1 实体与关系 18
4.4.2 溯源属性与节点共享 19
4.4.3 辅助分类层 20
4.4.4 图谱规模 20
4.5 数据存储设计 21
4.5.1 向量库设计 21
4.5.2 关系数据库设计 21
4.5.3 配置与日志文件 22
4.6 诊断流程设计 23
4.7 接口设计 24
4.8 本章小结 25
第 5 章 关键技术与实现 27
5.1 维修记录预处理 27
5.2 基于大模型的六元组抽取 27
5.2.1 抽取提示词设计 27
5.2.2 规则校验与专家复核 28
5.3 图谱规范化与入库 28
5.3.1 实体规范化 28
5.3.2 导入 Neo4j 29
5.4 统一检索的实现 29
5.4.1 问句解析 29
5.4.2 混合打分 30
5.4.3 图谱子图的选取 31
5.5 排查决策树生成 31
5.5.1 决策树的数据结构 31

5.5.2 大模型生成 32
5.5.3 规则生成与降级 33
5.5.4 成本等级的确定 33
5.6 前端实现 34
5.6.1 分阶段展示 34
5.6.2 知识图谱的单条链路高亮 34
5.6.3 决策树渲染 34
5.7 本章小结 35
第 6 章 测试与实验 36
6.1 实验环境与数据 36
6.2 功能测试 36
6.3 实验一：图谱构建质量（E1） 38
6.3.1 实验方法 38
6.3.2 实验结果 38
6.3.3 结果分析 39
6.4 实验二：检索效果（E2） 40
6.4.1 知识图谱子图检索（E2a） 40
6.4.2 相似记录检索（E2b） 41
6.4.3 交叉编码器精排（E2b 续） 42
6.5 实验三：排查决策树生成消融实验（E3） 42
6.5.1 实验设计 42
6.5.2 实验结果 43
6.5.3 结果分析 44
6.5.4 人工评价 45
6.5.5 提示词与证据格式的选型 46
6.6 实验四：系统性能（E4） 46
6.7 本章小结 47
第 7 章 总结与展望 49
7.1 工作总结 49
7.2 不足与展望 50
参考文献 52
致谢 55
附录 A 六元组抽检判定标准 56
附录 B 排查决策树人工评分标准 57

第 1 章 绪论

1.1 研究背景与意义

汽车零部件制造对生产节拍和设备稼动率要求很高。机加工车间的加工中心、清洗机、机械手等设备长时间连续运行，关键设备一旦停机，往往造成整条产线停线。设备故障后，维修人员要根据报警和故障现象，凭经验判断故障部位和原因，再逐项排查。排查效率很大程度上取决于个人经验：老员工能较快锁定问题，新员工则需要反复试错。

企业在长期生产中以电子表格形式积累了大量维修记录，每条记录包含设备、故障现象、原因和处理措施，是宝贵的维修经验。但这些记录难以利用：一是以口语化短文本为主，同一部件、同一故障常有多种写法；二是只能按关键词查找，看不出“设备—部件—故障—原因—措施”之间的因果关系；三是查到的是零散的原始记录，维修人员仍要自己归纳排查顺序。

知识图谱以实体和关系组织知识，能清晰表达设备、部件、故障、原因和措施之间的关联，便于追溯和可视化，在制造领域已有较多应用[1]，但它本身不擅长理解自然语言提问，也不能直接给出综合的处置建议。大语言模型语言理解和生成能力强，但缺少企业私有的设备知识，容易生成看似合理却与事实不符的内容，即“幻觉”[2]，直接用于工业现场存在风险。检索增强生成（RAG）在生成前先检索外部知识，用检索结果约束模型输出，是缓解这一问题的主要途径[3]；用知识图谱中的结构化知识增强大模型，也已成为研究热点[4]。

基于上述背景，本文依托某汽车零部件企业机加工车间的真实维修数据，设计并实现了一个基于知识图谱增强大模型的产线故障诊断系统：用大模型从历史维修记录中抽取故障知识，构建故障知识图谱；诊断时同时检索知识图谱子图和相似历史记录，把二者作为证据交给大模型，生成按原因分支、先易后难的排查决策树。

本文的意义在于：在技术上，探索知识图谱和相似记录两路证据共同增强大模型的方式，并在真实工业数据上通过消融实验定量分析各路证据的作用；在实践上，把分散在表格中的维修经验沉淀为可检索、可视化、可追溯的知识，帮助

维修人员快速了解同类故障的历史原因和处理方法，降低对个人经验的依赖。系统部署在企业内网，检索全部在本地完成，无网络时仍可用规则生成决策树，符合企业的数据安全和现场使用要求。

1.2 国内外研究现状

1.2.1 设备故障知识图谱

知识图谱在制造领域的应用已从语义检索扩展到决策支持，陶家琦等[1]将其归纳为语义关联、定性决策和综合决策三个层次。在设备故障诊断方面，刘晶等[5]针对小样本工况构建设备故障知识图谱，用元学习进行链接预测；周彬等[6]从冶金设备点检文档中抽取故障事件，构建因果知识图谱挖掘根因；陈宏等[7]用改进的 BERT-BiLSTM-CRF 抽取故障实体，按六元组本体在 Neo4j 中构建风电机组故障知识图谱；邢雪琪等[8]基于飞机故障隔离手册构建知识图谱并推荐维修方案。这些研究表明知识图谱能有效组织故障知识，但图谱构建大多依赖人工规则或专门训练的抽取模型，成本较高，用户交互多为关键词或模板查询。

1.2.2 维修文本知识抽取

维修记录篇幅短、口语化，缩写和设备编号多，是图谱构建的主要难点。田嘉鹏等[9]针对电力设备故障文本研究实体识别，党小超等[10]基于矿井提升系统的点检维修记录研究实体关系抽取，这类深度学习方法通常需要大量标注数据。大模型出现后，借助提示词进行零样本或少样本抽取成为新的途径[11]；Okazaki 等[12]用本体引导的大模型从多条产线的 FMEA 表单中抽取知识，构建统一的知识图谱。

1.2.3 检索增强生成与知识图谱增强大模型

Lewis 等[3]提出 RAG，将稠密检索器与生成模型结合。检索器方面，DPR[13]明显优于 BM25，BGE-M3[14]同时支持稠密、稀疏和多向量检索，适用于中文。Gao 等[15]和刘雪颖等[16]对 RAG 进行了综述；多路检索结果的融合常用倒数排名融合（RRF）[17]。知识图谱与大模型融合方面，Pan 等[4]提出了三类协同框架；GraphRAG[18]由大模型从文档构建图谱并生成社区摘要；Peng 等[19]把图检索增强

生成归纳为图索引、图引导检索和图增强生成三个阶段；冯拓宇等[20]综述了大模型增强的知识图谱问答。张鹤译等[21]、王永等[22]分别在中医方剂和科研管理领域构建了融合大模型与知识图谱的问答系统。

1.2.4 大模型在工业故障诊断中的应用

近两年大模型开始进入工业运维领域。程玉杰等[23]将航空电子装备故障知识图谱推理与子图增强的大模型推理交叉校验；邓养吾等[24]微调大模型生成电力故障抢修方案；Bahr 等[25]把 FMEA 数据构建为知识图谱，提出知识图谱增强的 RAG 框架；Ojima 等[26]针对汽车故障知识图谱优化 Graph RAG 的查询生成流程。

1.2.5 存在的问题

综合来看，该领域呈现出图谱构建由人工转向大模型辅助、检索由单路转向多路混合、输出由开放问答转向结构化决策、模型由云端转向私有化部署等趋势，但仍存在以下不足：（1）面向产线维修短文本的图谱自动构建质量不稳定，实体同义异写问题突出；（2）多数系统停留在问答层面，能输出可执行、可溯源排查流程的研究较少；（3）缺少基于真实工业数据、定量评估知识图谱给大模型带来多少增益的研究。

1.3 本文主要工作

本文以某机加工车间 2023—2025 年的 15319 条维修记录为数据基础，主要工作如下：

（1）故障知识图谱构建。设计了六类实体、五类关系的故障本体；用“字段定义＋难例规则＋输出约束”的提示词由大模型抽取六元组，经规则校验、专家复核和实体规范化后导入 Neo4j，构建了包含 6993 个实体节点、21919 条链路关系的故障知识图谱，每条关系保留来源记录编号，可追溯到原始记录。

（2）统一检索。实现了基于实体词典的问句解析，以及二元字组字面覆盖度与 BGE-M3 向量相似度加权的混合打分，一次检索得到的同一批结果同时用于图谱子图、相似记录和决策树。与旧版精确匹配相比，图谱子图检索的空结果率从 50.3%降到 7.3%；相似记录检索的 Hit@5 达到 95.8%。实验还发现通用的交叉编码器精排在本数据上没有带来提升，系统默认关闭。

（3）知识图谱增强的排查决策树生成。把图谱链路和相似记录组织成证据，由大模型生成按原因分支的 JSON 决策树，分支顺序由确定的成本规则重排；大模型不可用、超时或输出不合格时自动改用规则决策树。在 100 条测试问句上的消融实验表明，检索证据是大模型给出正确结果的前提（无证据时根因命中率仅 2%），图谱与相似记录合用时根因命中率最高（46%）。

（4）系统设计与实现。采用前后端分离的 B/S 架构，实现了分阶段返回的三段式诊断（知识图谱、相似记录、排查决策树）、知识图谱可视化与单条链路高亮、图谱编辑与回滚、数据导入、模型与提示词在线配置等功能。

（5）实验评估。从图谱构建质量、检索效果、决策树生成质量和系统性能四个方面进行了实验。

1.4 论文组织结构

全文共 7 章：第 1 章绪论；第 2 章介绍相关技术；第 3 章进行需求分析；第 4 章给出系统总体设计，包括架构、模块、知识图谱本体和数据存储；第 5 章介绍知识抽取、统一检索、决策树生成等关键技术的实现；第 6 章进行功能测试和实验评估；第 7 章总结全文并展望后续工作。

第 2 章 相关技术

本章介绍系统用到的主要技术：知识图谱及其存储、文本向量表示与检索、检索结果的融合与重排序、大语言模型与检索增强生成，以及系统开发所用的前后端框架。

2.1 知识图谱

2.1.1 知识图谱的基本概念

知识图谱用“实体—关系—实体”的三元组描述客观世界中的事物及其联系，实体作为图中的节点，关系作为节点之间的有向边，实体和关系都可以带有属性。与关系数据库按表存储不同，知识图谱直接以图的形式保存数据之间的关联，适合表达多跳的因果链和层级结构。在制造领域，知识图谱已用于工艺知识管理、质量追溯和设备故障诊断等场景[1]。

面向特定领域构建知识图谱时，通常先设计本体（schema），规定有哪些实体类型、实体之间允许哪些关系，再按本体从数据中抽取实例。设备故障领域常见的本体以“设备—部件—故障—原因—措施”为主线组织知识[7]，本系统也采用这一思路，在此基础上增加“区域”一层，形成六类实体、五类关系的故障本体（详见第 4 章）。

2.1.2 知识抽取

知识抽取是从非结构化文本中识别实体及实体之间关系的过程。传统方法依赖人工规则，或使用 BiLSTM-CRF、BERT 等模型做序列标注，需要大量人工标注数据[9][10]。大语言模型出现后，可以通过提示词说明字段定义和输出格式，让模型直接从文本中抽取结构化结果，不需要专门训练抽取模型[11]。这种方式标注成本低，但输出可能不稳定，需要配合格式校验和人工复核。本系统采用“大模型抽取 + 规则校验 + 专家复核”的方式构建六元组。

2.1.3 图数据库 Neo4j

Neo4j 是一种原生图数据库，采用属性图模型：节点可以有一个或多个标签（label），节点和关系都可以带键值对形式的属性。Neo4j 使用声明式查询语言 Cypher，用类似 ASCII 图形的语法描述要匹配的模式，例如(e:Equipment)-[:HAS_PART]->(c:Component)表示从设备节点沿 HAS_PART 关系到部件节点。按模式匹配多跳路径时，Neo4j 沿关系指针遍历，不需要像关系数据库那样做多表连接，查询子图的效率较高。导入数据时可以用 MERGE 语句实现“存在则复用、不存在则创建”，避免产生重复节点[27]。

2.2 文本向量表示与检索

2.2.1 基于词项的字面检索

字面检索按问句与文档共有的词项计算相关度，代表方法是 TF-IDF 和 BM25。其核心思想是：一个词在文档中出现越多、在整个语料中越少见，就越能代表该文档。逆文档频率（IDF）定义为

$$\mathrm{idf}(t) = \log \frac{N}{n_t} \tag{2-1}$$

式中，N 为语料中的文档总数，n_t 为包含词项 t 的文档数。字面检索对设备编号、部件名称等专有词非常敏感，但无法识别同义的不同说法。中文没有天然的词边界，分词错误会直接影响检索效果；以相邻两个字组成的二元字组（bigram）作为词项，可以绕开分词，对“刀库门”“Z 轴”这类领域词也有较好的覆盖。本系统的字面打分即采用二元字组加 IDF 加权的覆盖度（第 5 章）。

2.2.2 稠密向量检索

稠密向量检索用神经网络编码器把问句和文档分别映射为固定维度的向量，用向量之间的余弦相似度衡量语义相关度：

$$\cos(\boldsymbol{q}, \boldsymbol{d}) = \frac{\boldsymbol{q} \cdot \boldsymbol{d}}{\| \boldsymbol{q} \| \, \| \boldsymbol{d} \|} \tag{2-2}$$

由于问句和文档各自独立编码（双塔结构），文档向量可以离线预先计算，检索时只需编码问句，再在向量库中做近邻搜索，速度快，适合大规模召回。DPR 证明了稠密检索在开放域问答中可以明显优于 BM25[13]。BGE-M3 是北京智

源研究院发布的多语言嵌入模型，支持中文，输出 1024 维稠密向量，同时支持稀疏和多向量检索，最长可处理 8192 个词元的输入[14]。本系统用 BGE-M3 对维修记录进行向量化。

2.2.3 向量数据库 Qdrant

向量数据库专门用于存储向量并执行近似最近邻搜索。Qdrant 是一个开源向量数据库，以集合（collection）组织数据，每条数据（point）包含一个向量和一个 JSON 格式的负载（payload）；检索时可以在向量相似度搜索的同时按 payload 字段过滤，例如只在指定设备的记录中检索。Qdrant 采用 HNSW（分层可导航小世界图）索引实现近似最近邻搜索[28]。

2.3 检索结果融合与重排序

2.3.1 加权融合与倒数排名融合

字面检索擅长精确匹配专有词，向量检索擅长识别语义相近的不同说法，二者结合通常好于单独使用其中一种。常见的融合方式有两种。一种是分数加权，把两路分数归一化到同一区间后按权重相加（本系统的做法见第 5 章式（5-4））；另一种是倒数排名融合（Reciprocal Rank Fusion，RRF）[17]，只使用各路检索中的名次而不使用原始分数：

$$\mathrm{RRF}(d) = \sum_i \frac{1}{k + r_i(d)} \qquad (2\text{-}3)$$

式中，$r_i(d)$为文档 d 在第 i 路检索结果中的名次，k 为平滑常数，常取 60。RRF 不受各路分数尺度不同的影响，不需要调权重，是多路检索融合中常用的基线方法。第 6 章的检索实验对这两种融合方式都做了对比。

2.3.2 交叉编码器重排序

双塔结构的向量检索为了速度，让问句和文档在编码时互不可见，相关度判断较粗。交叉编码器（cross-encoder）把问句和候选文档拼接成一个输入，由 Transformer 模型直接输出相关度分数，二者的每个词元之间都能充分交互，判断更精细，但每个候选都要单独计算一次，开销大，只适合对少量候选做重排序。

因此常见的做法是“召回—精排”两阶段：先用快速方法召回几十条候选，再用交叉编码器精排。bge-reranker 是与 BGE 系列配套的交叉编码器重排序模型[29]。

需要注意的是，通用领域训练的重排序模型对输入格式比较敏感：问句和文档的表述方式差别很大时，模型可能偏向信息量多的长文本。第 6 章的实验表明，在本系统的维修记录上，问句与记录的输入格式不对齐会导致精排反而降低命中率。

2.4 大语言模型与检索增强生成

2.4.1 大语言模型

大语言模型（Large Language Model，LLM）以 Transformer 解码器为基本结构，在海量文本上以预测下一个词元为目标进行预训练，再经过指令微调和人类反馈对齐，具备理解自然语言指令、按要求生成文本的能力。Qwen3 等开源模型可以在企业内部私有化部署[30]；企业也可以通过统一的模型服务提供与 OpenAI 兼容的调用接口，应用程序只需改变服务地址和模型名称即可切换模型。

大模型的主要问题是“幻觉”，即生成看似合理但与事实不符的内容[2]。在故障诊断场景中，通用大模型不了解企业设备的具体情况，可能编造不存在的部件或不合适的处理方法，直接使用存在风险。

2.4.2 提示词工程与结构化输出

提示词（prompt）是传给大模型的输入文本，包括任务说明、约束条件、参考资料和输出格式要求等。通过在提示词中明确字段定义、给出判定规则和难例、规定只输出 JSON，可以让大模型的输出更稳定、便于程序解析。生成时把采样温度设为 0，可以减小输出的随机性，便于复现。即便如此，大模型的输出仍可能不符合格式要求，程序端需要做格式和内容校验，校验失败时采用兜底方案。

2.4.3 检索增强生成

检索增强生成（RAG）在生成前先从外部知识库检索与问题相关的资料，把资料和问题一起交给大模型，要求模型依据资料作答[3]。这样既能让模型使用私有的、最新的知识，又能约束模型少编造，并且答案可以追溯到具体资料。RAG

的基本流程包括建立索引、检索和生成三个环节，研究者在此基础上发展出了查询改写、多路检索、重排序等改进方法[15][16]。

2.4.4 知识图谱增强的大模型

普通 RAG 检索的是文本片段，片段之间的关联需要大模型自己归纳。知识图谱以结构化方式保存实体之间的关系，把图谱中的相关子图作为检索结果交给大模型，可以直接提供“某故障由哪些原因引起、各用什么措施处理”这样的因果链，弥补文本检索缺少结构的不足[4][19]。GraphRAG 等方法用大模型从文档构建图谱再进行检索[18]；在工业领域，已有研究把 FMEA 数据或汽车故障数据构建为知识图谱，用于增强大模型的问答和分析[25][26]。本系统同时检索知识图谱子图和相似历史记录，把两路证据一并交给大模型生成排查决策树，并通过消融实验检验两路证据各自的作用。

2.5 系统开发技术

2.5.1 后端：FastAPI 与服务端推送

FastAPI 是基于 Python 类型注解的 Web 框架，支持异步处理，能根据接口定义自动生成接口文档。本系统后端用 FastAPI 提供 REST 接口，诊断对话接口采用服务器发送事件（Server-Sent Events，SSE）流式返回：服务器在一个 HTTP 连接上连续推送多个 data:帧，前端每收到一帧就更新对应区域。这样，知识图谱和相似记录在检索完成后即可先显示，不必等大模型生成决策树（约 40 秒，见第 6 章）结束。会话和消息记录保存在轻量级嵌入式数据库 SQLite 中。

大模型通过 LangChain 的 OpenAI 兼容客户端调用，既可以连接企业内部的大模型网关，也可以切换为本地 Ollama 服务。LangGraph 是 LangChain 生态中用状态图编排多步骤流程的框架；本系统用它定义了“路由—诊断—生成”的非流式处理流程，流式接口则按同样的步骤在程序中依次调用。

2.5.2 前端：React 与可视化组件

前端采用 React 构建单页应用，用 Vite 作为开发和打包工具，界面样式使用 Tailwind CSS。知识图谱用 Cytoscape.js 绘制，它是一个开源的图可视化库，支

持力导向等多种布局，并能响应节点点击、悬浮等交互事件；本系统在此基础上实现了“点击部件，高亮同一条维修记录对应的完整链路”。排查决策树用 Mermaid 渲染：程序把决策树转换成 Mermaid 的流程图描述文本，由 Mermaid 在浏览器中绘制成流程图。

2.6 本章小结

本章介绍了系统涉及的主要技术。知识图谱和 Neo4j 用于组织、存储故障知识；字面检索、BGE-M3 向量检索和 Qdrant 用于查找相似历史记录，加权融合、RRF 和交叉编码器重排序用于改进检索排序；大语言模型在检索增强生成框架下，依据知识图谱和历史记录两路证据生成排查决策树。下一章在此基础上分析系统需求。

第 3 章 需求分析

3.1 业务现状与问题

某汽车零部件企业机加工车间有加工中心、清洗机、机械手等设备约一百台，分布在三个生产区域。设备发生故障时，维修工程师到现场查看报警和故障现象，凭经验判断故障部位和原因，再逐项排查、处理，处理完成后把故障现象、原因和处理措施登记到维修记录表中。2023—2025 年共积累维修记录 15319 条。

现有做法存在以下问题：

（1）排查依赖个人经验。经验丰富的工程师能较快锁定原因，新员工往往需要反复试错，停机时间长。

（2）历史记录难以利用。维修记录是口语化的短文本，同一部件、同一故障有多种写法（如“刀库门”“刀库防护门”），在表格中只能按关键词查找，查不全，也看不出“设备—部件—故障—原因—措施”之间的对应关系。

（3）查到的记录需要人工归纳。即使找到十几条相似记录，工程师仍要自己总结有哪些可能原因、先查哪个后查哪个。

（4）现场网络不稳定。车间部分区域网络信号差，依赖外部服务的工具在断网时无法使用。

3.2 业务流程分析

引入系统后的故障排查流程如图 3-1 所示：工程师在系统中输入故障描述（可带设备编号），系统依次返回相关的知识图谱子图、相似历史记录和排查决策树；工程师按决策树逐个分支排查，必要时继续追问；故障处理后，新的维修记录由管理员定期导入，经抽取、复核后补充进知识图谱和向量库，形成“使用—积累—再使用”的闭环。

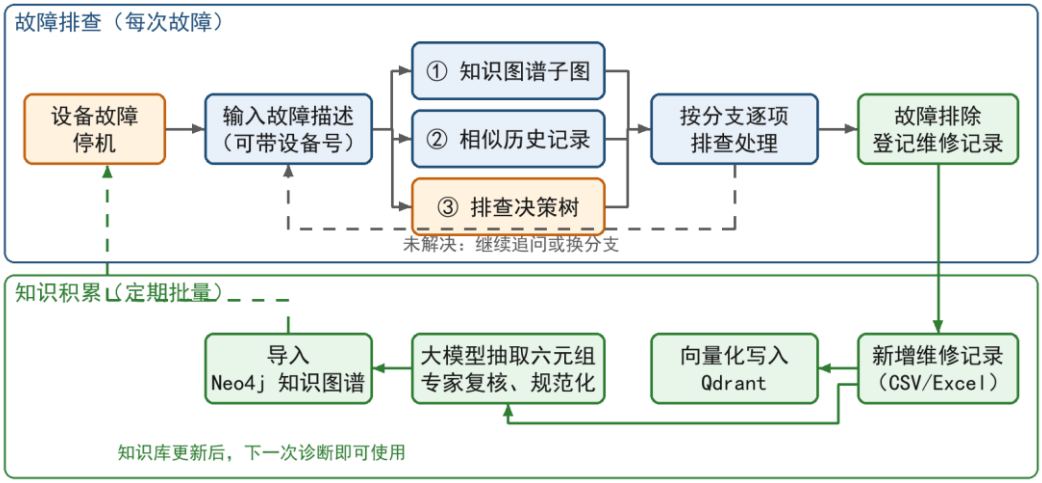


图 3-1 引入系统后的故障排查业务流程

3.3 用户角色

根据系统的使用方式，用户分为三类：

（1）维修工程师（只读用户）：系统的主要使用者，输入故障描述获得诊断结果，查看知识图谱和相似记录，按排查决策树处理故障，可以追问、导出报告、收藏流程图。

（2）知识工程师：在维修工程师权限之外，负责维护知识图谱，修改错误的节点和关系，合并同义实体。

（3）管理员：在知识工程师权限之外，负责导入维修数据、回滚错误的图谱修改、配置大模型和提示词、设置诊断数据源开关。

此外，系统还与两类外部对象交互：大模型服务（企业内部模型网关，或本地部署的开源模型），以及存放知识的图数据库和向量数据库。

系统用例如图 3-2 所示。

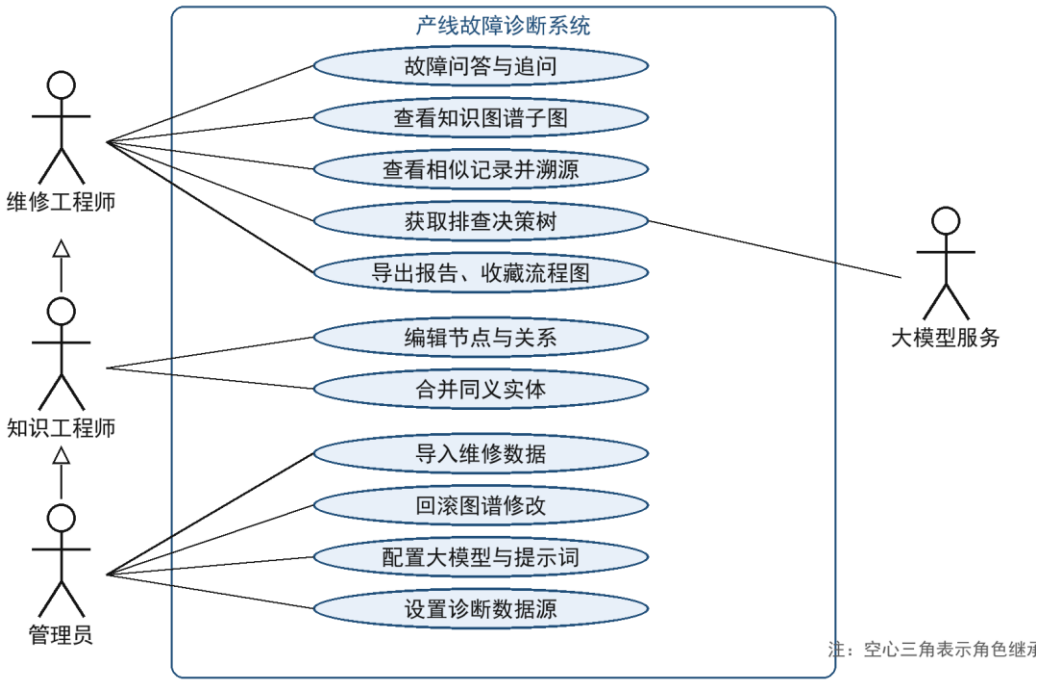


图 3-2 系统用例图

3.4 功能需求

3.4.1 故障诊断

（1）故障问答：用户用自然语言描述故障，问句中可以带设备编号，也可以只写故障现象；支持中英文；同一会话中可以对诊断结果继续追问。

（2）知识图谱子图展示：识别问句中的设备和部件，返回相关的“区域—设备—部件—故障—原因—措施”链路，以图的形式展示；点击某个部件时，高亮这条维修记录对应的完整链路；节点可以追溯到来源维修记录。

（3）相似记录展示：列出与问句最相似的历史维修记录，显示工位、问题、原因、措施和相似度，每条记录可以追溯到来源文件和行号。

（4）排查决策树生成：综合知识图谱和相似记录，生成按原因分支的排查决策树，每个分支给出可能原因、处理步骤和依据；分支按处理成本由低到高排列，先查容易的、后查麻烦的。决策树中标明是由大模型生成还是由规则生成。

（5）分阶段返回：知识图谱和相似记录检索完成后立即显示，不必等待决策树生成。

（6）断网可用：无法连接大模型服务时，系统仍要给出排查决策树。

（7）结果保存与导出：诊断会话自动保存，可以查看和删除历史会话；排查流程图可以收藏到流程图库；诊断结果可以导出为 PDF、Word 或图片。

3.4.2 知识管理

（1）维修数据导入：导入 CSV 或 Excel 格式的维修记录，完成向量化并写入向量库；导入六元组链路表，写入知识图谱。

（2）知识图谱浏览：按区域、设备、部件、故障、原因、措施逐级筛选浏览，查看节点数、关系数等统计信息。

（3）知识图谱编辑：修改节点名称和属性，增删关系，把同义的节点合并为一个。

（4）修改回滚：每次修改都记录日志，可以撤销单次修改或整组修改。

3.4.3 系统配置

（1）大模型配置：选择大模型服务类型，填写服务地址、密钥和模型名称，并能测试连接。

（2）提示词管理：在线修改各环节使用的提示词，保留历史版本，可以恢复到旧版本或默认版本。

（3）诊断数据源配置：可以分别开关知识图谱、相似记录和精排，选择决策树生成方式（自动或只用规则），并提供“模拟断网”开关。这组开关既用于现场按需调整，也用于第 6 章的消融实验。

（4）界面语言：支持中文和英文切换。

3.5 非功能需求

（1）响应时间：检索阶段（知识图谱和相似记录）应在 2 秒内返回；大模型生成决策树受外部服务影响，一般应在 1 分钟内完成，并设置超时，超时后改用规则决策树；无网络时整个诊断应在数秒内完成。

（2）数据安全：系统部署在企业内网，维修数据、知识图谱和向量库都保存在本地；大模型只调用企业内部的模型服务，生产数据不出内网；论文、截图和演示中使用脱敏数据。

（3）可追溯性：知识图谱中的每条关系、表格中的每条记录都要能追溯到

原始维修记录，便于工程师判断结论是否可信。

（4）可靠性：大模型调用失败、超时或输出不符合格式时，系统自动改用规则决策树，保证始终有可用结果。

（5）可维护性：数据源开关、大模型和提示词都可在界面上配置，修改后下一次提问即生效，不需要改代码或重启服务。

（6）易用性：界面简洁，现场工程师无需培训即可使用；支持在普通办公电脑的浏览器中访问。

3.6 可行性分析

（1）技术可行性：系统用到的 Neo4j、Qdrant、BGE-M3、FastAPI、React 等都是成熟的开源技术；大模型通过与 OpenAI 兼容的接口调用，可以对接企业已有的模型服务，也可以换成本地部署的开源模型。

（2）数据可行性：已获得 2023—2025 年的 15319 条真实维修记录，其中 2023 年的 5455 条已完成六元组抽取和专家复核，知识图谱和向量库均已建立。

（3）经济可行性：所用软件均为开源软件，图数据库、向量数据库和嵌入模型可在普通工作站（本文实验环境为 7.3 GB 内存的笔记本电脑）上运行，大模型使用企业已有服务，不需要额外购买显卡等硬件。

（4）操作可行性：系统通过浏览器访问，输入方式与日常搜索一致，现场工程师容易上手。

3.7 本章小结

本章分析了车间设备维修的现状和问题，梳理了引入系统后的业务流程，确定了维修工程师、知识工程师和管理员三类用户，从故障诊断、知识管理和系统配置三方面提出了功能需求，并给出了响应时间、数据安全、可追溯性等非功能需求，最后从技术、数据、经济和操作四个方面论证了可行性。

第 4 章 总体设计

4.1 设计原则

根据第 3 章的需求，系统设计遵循以下原则：

（1）本地优先：知识图谱、向量库、嵌入模型都部署在本地，检索不依赖外网；只有决策树生成调用大模型。

（2）结果可降级：大模型不可用时，用规则生成同样格式的决策树，保证诊断始终有结果。

（3）证据可追溯：图谱中的每条关系和表格中的每条记录都带有来源记录编号。

（4）检索结果共用：一次检索得到的同一批候选记录，同时用于图谱展示、记录表和决策树生成，三处结果彼此一致。

（5）配置与代码分离：数据源开关、大模型和提示词都保存在配置文件中，在界面上修改后下一次提问即生效。

4.2 系统架构

系统采用前后端分离的 B/S 架构，分为四层（图 4-1）：

（1）表示层：基于 React 的单页应用，包括诊断对话、数据管理、流程图库、系统设置等页面，通过 HTTP 与后端通信，诊断结果通过 SSE 流式接收。

（2）接口层：FastAPI 提供的 REST 接口和 SSE 流式接口，负责请求解析、会话管理和结果推送。

（3）业务层：包括统一检索模块、决策树生成模块、知识图谱管理模块、数据导入模块和配置管理模块。

（4）数据层：Neo4j 保存故障知识图谱，Qdrant 保存维修记录向量，SQLite 保存会话、消息和收藏的流程图，配置和图谱修改日志以文件形式保存。外部的大模型服务通过 OpenAI 兼容接口接入，可以是企业内部的模型网关，也可以是本地 Ollama 服务。

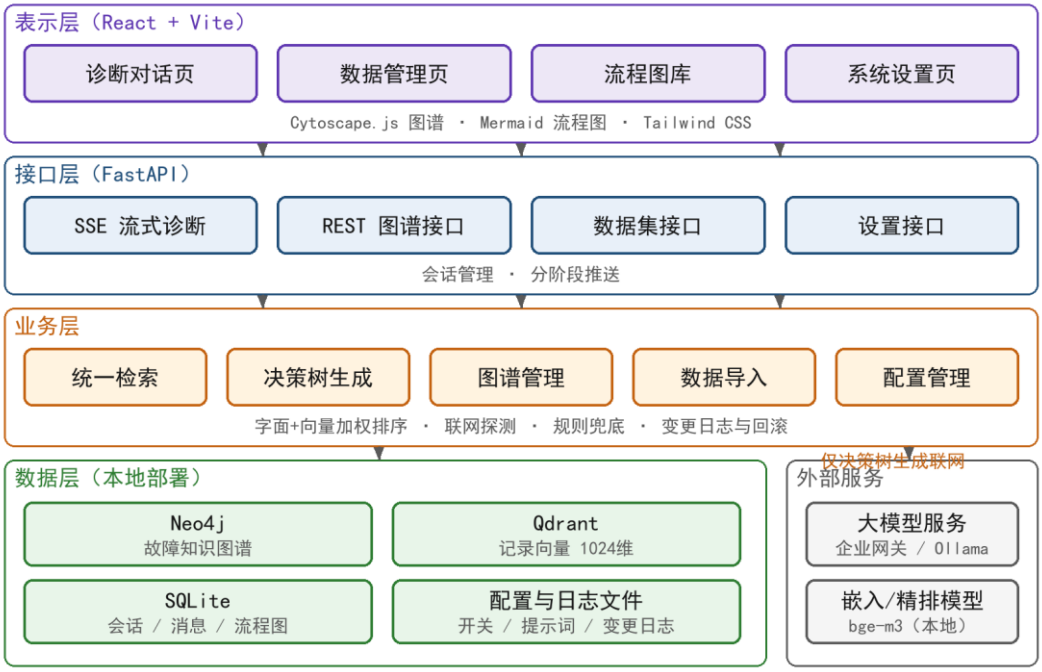


图 4-1 系统总体架构

4.3 功能模块设计

系统功能模块如图 4-2 所示。

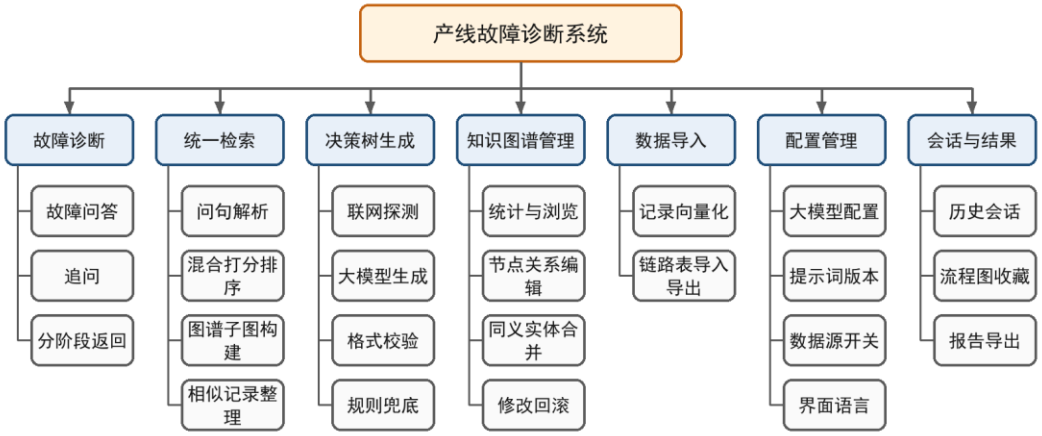


图 4-2 系统功能模块

（1）故障诊断模块：接收问句，判断是新问题还是追问。新问题依次完成问句解析、统一检索、图谱子图构建、相似记录整理和决策树生成，按阶段把结果推送给前端；追问则基于本会话已有的检索结果由大模型回答。

（2）统一检索模块：识别问句中的设备、区域和品牌，缩小候选范围；对

候选记录计算字面覆盖度和向量相似度并加权排序（可选交叉编码器精排）；从排序结果中选出节点名与问句对得上的链路构成图谱子图，同时生成相似记录表和规则决策树。

（3）决策树生成模块：先探测大模型服务能否连通。连通时把图谱链路和前 6 条相似记录组织成证据，按提示词要求大模型输出 JSON 格式的决策树，校验通过后按成本等级排序分支；不连通、调用超时或输出不合格时，改用规则决策树。

（4）知识图谱管理模块：提供图谱统计、逐级浏览、节点和关系的增删改、同义节点合并；每次修改写入变更日志并记录逆操作，用于回滚。

（5）数据导入模块：把 CSV 或 Excel 格式的维修记录向量化后写入 Qdrant；导入、导出六元组链路表，写入或导出 Neo4j。

（6）配置管理模块：管理大模型连接参数、提示词及其历史版本、诊断数据源开关和界面语言。

（7）会话与结果管理模块：保存会话和消息，支持历史会话的查看和删除、流程图收藏和诊断报告导出。

4.4 故障知识图谱本体设计

4.4.1 实体与关系

根据维修记录的结构和排查业务的逻辑，设计了包含六类实体、五类关系的故障本体（图 4-3，表 4-1、表 4-2）。

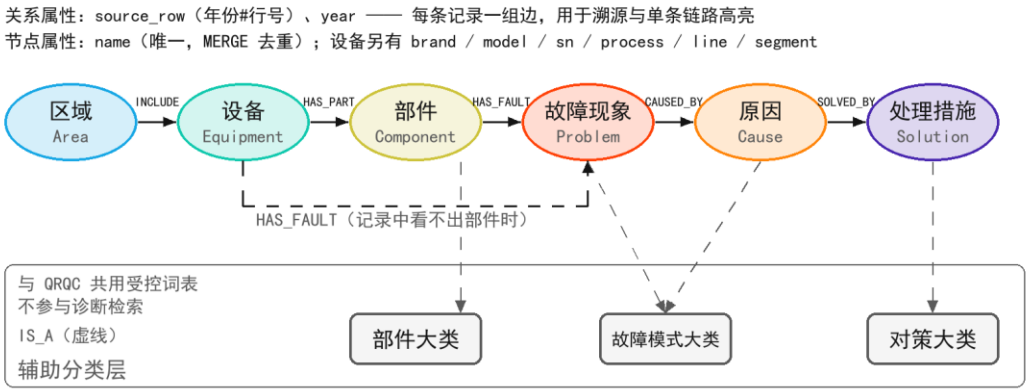


图 4-3 故障知识图谱本体

表 4-1 故障知识图谱的实体类型

实体类型	标签	含义	来源	示例
区域	Area	生产区域	记录字段	机加区 A
设备	Equipment	设备（工位）	记录字段	EQ-001
部件	Component	发生故障的部件	大模型抽取	刀库门
故障现象	Problem	故障的表现	大模型抽取	刀库门打不开
原因	Cause	导致故障的原因	大模型抽取	气缸漏气
处理措施	Solution	最终有效的处理动作	大模型抽取	更换气缸

表 4-2 故障知识图谱的关系类型

关系	起点 → 终点	含义
INCLUDE	区域 → 设备	区域包含设备
HAS_PART	设备 → 部件	设备有某部件（故障）
HAS_FAULT	部件 → 故障现象	部件出现某故障；原始记录看不出部件时由设备直接指向故障现象
CAUSED_BY	故障现象 → 原因	故障由某原因引起
SOLVED_BY	原因 → 处理措施	原因用某措施处理

一条维修记录对应一条“区域—设备—部件—故障现象—原因—处理措施”链路，即一个六元组。

4.4.2 溯源属性与节点共享

所有节点以名称（name）作为唯一标识，导入时用 MERGE 去重，同名实体共用一个节点，因此不同记录中相同的部件、故障会汇聚到同一节点上，便于看出“同一故障有哪些不同原因”。但节点共享后，仅从图结构上无法区分哪条边属于哪条记录。为此，在每条关系上保存来源属性 source_row（格式为“年份#行号”）和 year，并把 source_row 作为关系唯一性的一部分：每条维修记录在链路上各有一组自己的边，同一对节点之间可以有多条来自不同记录的平行边。返回给前端时，平行边合并为一条显示，边上带所有来源编号的列表。前端点击部件

时，沿来源编号相同的边遍历，就能只高亮属于同一条维修记录的完整链路；记录表中的每条记录也可以通过同一编号回到原始数据。

设备节点上另外保存台账属性（品牌、型号、序列号、工序、产线、区段），悬浮查看设备节点时显示。

4.4.3 辅助分类层

为了与企业的质量问题管理（QRQC）系统共用一套受控词表，图谱中另外增加了一层分类节点：部件大类（PartClass）、故障模式大类（FailureMode）和对策大类（Countermeasure），具体的部件、故障现象/原因、处理措施通过 IS_A 关系挂到对应大类上，大类之间用 SUBCLASS_OF 表示层级。这一层只是附加的索引，不改动六元组链路，诊断检索不依赖它。

4.4.4 图谱规模

当前图谱由 2023 年经专家复核的 5454 条六元组构建，规模如表 4-3 所示（2026 年 11 月 11 日统计）。

表 4-3 故障知识图谱规模

类型	数量	类型	数量
区域 Area	3	链路关系（5 类）合计	21919
设备 Equipment	99	其中 INCLUDE	103
部件 Component	775	HAS_PART / HAS_FAULT / CAUSED_BY / SOLVED_BY	各 5454
故障现象 Problem	1717	分类关系 IS_A	6891
原因 Cause	1482	分类关系 SUBCLASS_OF	5
处理措施 Solution	2917	分类节点（PartClass 40、FailureMode 29、Countermeasure 13）	82
六元组实体节点合计	6993	全部节点 / 全部关系	7075 / 28815

4.5 数据存储设计

4.5.1 向量库设计

每条维修记录在 Qdrant 中存为一个点：向量为 BGE-M3 对记录文本编码得到的 1024 维稠密向量，距离度量为余弦相似度；负载字段见表 4-4。按年份和数据批次分为多个集合，检索时可在设置中选择启用哪些集合。检索时用 station 字段过滤，只在问句识别出的设备的记录中检索。

表 4-4 向量库负载字段

字段	含义
record_id	记录编号
line / station	区域（产线）/ 设备（工位）
problem / cause / action	故障现象 / 原因 / 处理措施
plan	后续计划
date	日期
source_year / source_row / source_file	来源年份 / 来源行号 / 来源文件，与图谱的 source_row 对应
source_row_hash	记录内容摘要，用于去重

4.5.2 关系数据库设计

会话、消息和收藏的流程图保存在 SQLite 中，表结构见表 4-5 至表 4-7。

表 4-5 会话表 conversations

字段	类型	说明
conversation_id	TEXT，主键	会话编号
title	TEXT	会话标题（取第一个问句）
message_count	INTEGER，默认 0	消息数
last_activity	TEXT（ISO 时间），非空	最后活动时间
created_at	TEXT（ISO 时间），非空	创建时间

表 4-6 消息表 messages

字段	类型	说明

字段	类型	说明
id	INTEGER，主键，自增	消息编号
conversation_id	TEXT，外键（级联删除），建索引	所属会话
role	TEXT	user / assistant
content	TEXT	消息文本
timestamp	TEXT（ISO 时间）	时间
qdrant_records	TEXT（JSON）	本轮检索到的相似记录，供追问时复用
extra_payload	TEXT（JSON）	图谱子图、决策树等结构化结果

表 4-7 流程图收藏表 flowcharts

字段	类型	说明
id	TEXT，主键	编号
title / query	TEXT	标题 / 原始问句
mermaid_code	TEXT	流程图的 Mermaid 描述
flow_plan	TEXT（JSON）	决策树结构
records	TEXT（JSON）	生成时依据的记录
created_at	TEXT（ISO 时间）	收藏时间

SQLite 没有专门的日期类型，时间以 ISO 8601 格式的文本保存；JSON 字段同样以文本保存。

4.5.3 配置与日志文件

（1）诊断数据源配置 diagnosis_config.json：use_kg（知识图谱）、use_vector（相似记录）、use_rerank（精排，默认关闭）、flow_generator（auto/rule）、simulate_offline（模拟断网）。

（2）提示词配置 prompts_config.json，每次保存时在 prompts_history/中留存历史版本。

（3）图谱变更日志 graph_mutations.jsonl：每行一次修改，记录编号、时间、操作类型、操作对象、请求内容和逆操作。回滚时执行逆操作，逆操作分为删除节点、恢复节点状态、重建节点、删除关系、重建关系五种；同一次用户操作产

生的多条修改共用一个组编号，可以整组回滚。

4.6 诊断流程设计

一次新问题的诊断流程如图 4-4 所示：

（1）接收问句，判断是否为追问；英文问句先翻译为中文。

（2）统一检索：解析问句中的设备、区域、品牌；在候选记录上计算字面覆盖度和向量相似度，按 0.65:0.35 加权排序；选出与问句对得上的图谱链路，整理相似记录，并生成规则决策树。

（3）推送知识图谱子图（diagnosis_kg）和相似记录（diagnosis_records），前端立即显示。

（4）决策树生成：若配置为只用规则或没有检索结果，直接采用规则决策树；否则探测大模型服务（3 秒内能否建立 TCP 连接，结果缓存 30 秒；“模拟断网”开关打开时直接判为不连通）。连通时调用大模型生成决策树（超时 45 秒），校验通过则按成本等级排序分支；不连通、超时或输出不合格时采用规则决策树，并记录原因。

（5）推送决策树（diagnosis_flowchart），结果中带生成方式（大模型/规则）和改用规则的原因；保存会话后推送结束标志（done）。

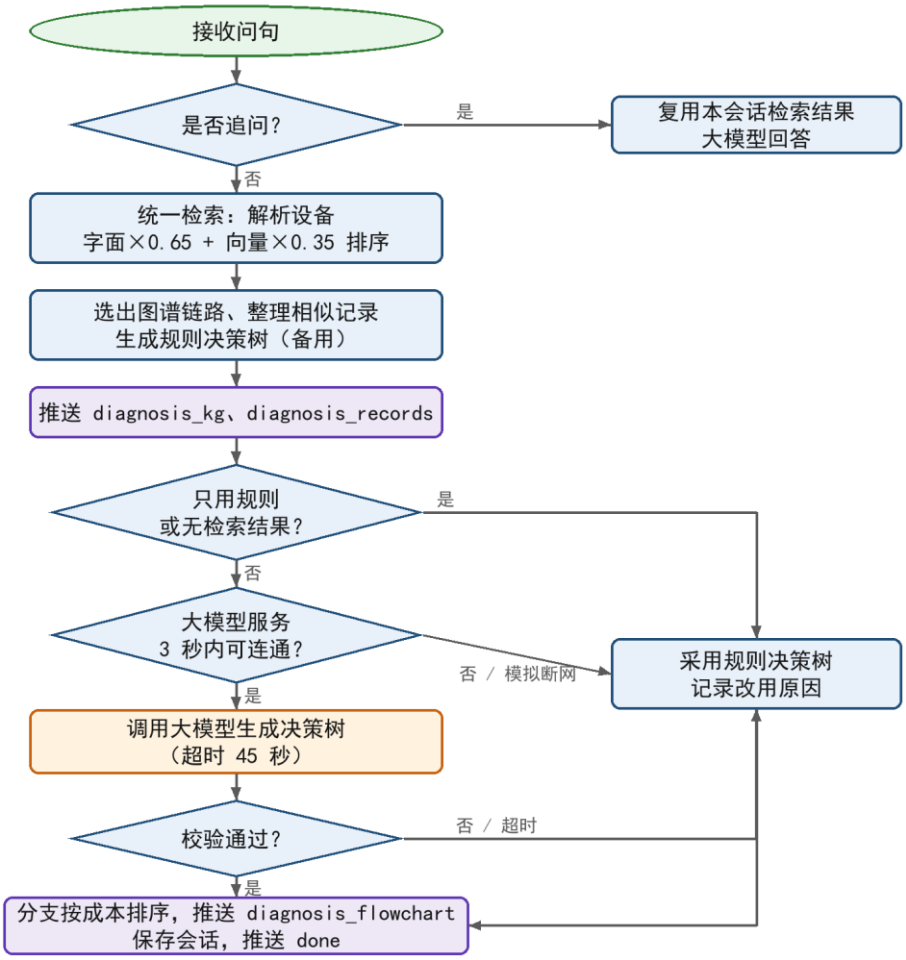


图 4-4 诊断流程

4.7 接口设计

后端主要接口见表 4-8。

表 4-8 主要接口

分类	方法与路径	说明
诊断	POST /chat	诊断对话，SSE 流式返回
会话	GET /conversations；GET /conversations/{id}/messages；DELETE /conversations/{id}	会话列表、消息、删除
图谱查询	GET /graph/overview、/graph/search、/graph/neighbors	图谱浏览与检索

分类	方法与路径	说明
图谱编辑	POST /graph/node；PUT、DELETE /graph/node/{id}；POST /graph/node/merge；POST /graph/edge；DELETE /graph/edge/{id}	节点、关系的增删改与合并
回滚	GET /graph/mutations；POST /graph/mutations/{id}/rollback；POST /graph/mutations/{group_id}/rollback-group	修改日志与回滚
图谱导入导出	GET /graph/export-csv；POST /graph/import-csv	六元组链路表
数据集	POST /api/datasets/embed；GET /api/datasets/collections 等	记录向量化与集合管理
流程图	GET、POST /flowcharts；GET、DELETE /flowcharts/{id}	流程图收藏
设置	GET、POST /settings/diagnosis、/settings/prompts、/settings/llm；POST /settings/llm/test	配置管理

SSE 接口每一帧为一个 JSON 对象，用 type 字段区分内容，主要类型见表 4-9。

表 4-9 诊断接口的推送帧类型

type	内容
meta	会话编号等元信息
progress	处理进度提示
diagnosis_kg	知识图谱子图（节点、边、来源编号）
diagnosis_records	相似记录列表
diagnosis_flowchart	决策树（结构、生成方式、改用规则的原因）
content	文本回答（追问时）
done / error	结束 / 出错

4.8 本章小结

本章给出了系统的总体设计：采用四层 B/S 架构，划分了故障诊断、统一检索、决策树生成、知识图谱管理、数据导入、配置管理和会话管理七个模块；设计了六类实体、五类关系的故障本体，用关系上的来源编号实现节点共享下的单

条链路溯源；设计了向量库、关系数据库和配置日志文件的存储结构；给出了诊断流程和接口设计。下一章介绍关键技术的实现。

第 5 章 关键技术与实现

本章介绍系统中几项关键技术的具体实现：维修记录预处理与六元组抽取、图谱规范化与入库、统一检索、排查决策树生成，以及前端的分阶段展示与链路高亮。

5.1 维修记录预处理

原始维修记录来自车间 2023—2025 年的三个 Excel/CSV 台账，共 15319 条（2023 年 5455 条、2024 年 5248 条、2025 年 4616 条）。三年的表格列名、编码不完全一致（2024、2025 年为 GBK 编码），预处理包括：

（1）字段映射：统一为区域（line）、设备（station）、故障现象（problem）、故障原因（cause）、处理措施（action）、后续计划（plan）、日期（date）等字段，并为每条记录生成来源编号 source_row（年份#行号），后续图谱、向量库和界面都用这个编号互相对应。

（2）设备编号归一化：同一台设备有“PM18-A/B1”“PM18-AB1”等写法，比较时去掉空格、横线、斜杠、井号并转为大写。

（3）脱敏：为论文和演示单独生成脱敏数据，设备编号按区域顺序替换为 EQ-001 等代号，区域替换为“机加区 A/B/C”，人名删除，日期精确到月，设备品牌替换为“品牌 X”；映射表只保存在本地，不随代码和论文外传。

5.2 基于大模型的六元组抽取

5.2.1 抽取提示词设计

区域和设备直接从记录字段复制，部件、故障现象、原因和处理措施由大模型从“故障现象描述”“解决对策”“故障原因”三列文本中抽取。提示词采用“字段定义 + 难例规则 + 输出约束”的结构：

（1）字段定义与长度限制：故障现象不超过 10 个汉字，只保留核心现象；原因不超过 8 个汉字，写成“部件+失效模式”（如磨损、松动、堵塞）；处理措施不超过 12 个汉字，只保留一个关键动作，去掉排查、测试确认、备件来源等

过程。要求故障现象与原因不能同义——现象是“看到的”，原因是“为什么”。

（2）部件抽取的优先顺序：故障描述中出现部件名词时优先取描述中的部件；描述中只有现象词（漏气、报警）时，才从处理措施中取被维修的部件；两者都没有时，填“设备_故障描述”。

（3）难例规则：动作词和工序词（换刀、移动、定位等）不能作为部件；“拆钣金、装钣金”只是维修过程，不是故障部件，应取真正被更换或调整的部件；报警码类故障取报警码对应的轴号；不能只返回站号。

（4）输出约束：只输出包含四个字段的 JSON，所有字段不能为空，采样温度为 0。

模型输出后，程序提取回复中的第一个完整 JSON 对象；部件为空或只有站号时，附加“上次返回了空或纯站号，必须取真正的部件名”的提示重试一次；仍失败时，用约 80 个常见部件名组成的词表从描述和对策中匹配部件兜底。

5.2.2 规则校验与专家复核

抽取完成后，用规则检查空值、字段超长、“故障现象与原因完全相同”、部件只有站号等问题。2023 年 5455 条记录中，5454 条抽取成功，1 条用词表兜底；各字段无空值；部件采用“设备_故障描述”格式的有 37 条，故障现象与原因完全相同的有 36 条，均列入复核清单。随后由维修专家逐条复核并修正，复核后的六元组作为构建图谱的依据。抽取质量的人工抽检结果见第 6 章。

5.3 图谱规范化与入库

5.3.1 实体规范化

抽取结果中同一实体常有不同写法，例如“丝杠异响”与“Y 轴丝杠异响”、“报警撞击”与“报警撞机”，若不处理，会在图谱中形成大量只连一条记录的孤立节点，同一故障的不同原因也无法汇聚到一起。规范化分两步：先用规则处理确定性的情况（错别字、轴号缺失、括号和空格差异等），再把同一部件下名称相近的候选交给大模型判断是否同义，并结合上下文（同一部件）给出统一名称。所有修改都记录到变更清单中，包括来源编号、字段、原值、新值、上下文和依据，便于审核和撤销。2023 年数据共产生 1258 处修改，其中故障现象 931 处、处理

措施 176 处、原因 151 处；依据为大模型判断的 1202 处，规则判断的 54 处，人工确认的 2 处。

5.3.2 导入 Neo4j

规范化后的六元组按“区域—设备—部件—故障现象—原因—处理措施”导入 Neo4j。节点用 MERGE 按名称去重，同名实体共用一个节点；关系则把来源编号 source_row 作为 MERGE 的一部分，每条记录生成自己的一组边，年份记入 year 属性。以部件到故障现象一段为例：

```
MERGE (c:Component {name: $c})
MERGE (p:Problem {name: $p})
WITH c, p
MERGE (c)-[r:HAS_FAULT {source_row: $src}]->(p)
SET r.year = $year
```

这样重复导入同一条记录不会产生重复的边，而不同记录在同一对节点之间各有一条边，关系总数等于记录数（HAS_PART、HAS_FAULT、CAUSED_BY、SOLVED_BY 各为 5454 条，表 4.3）。设备到区域的 INCLUDE 关系不区分记录，每台设备只有一条。

导入后，图谱规模见表 4-3。

5.4 统一检索的实现

统一检索模块（unified_retrieval.py）在系统启动时把三年的全部记录（15319 条）与图谱链路（5454 条）按来源编号合并成一个内存索引，每次检索只做一次打分排序，得到的同一批候选同时用于图谱、记录表和规则决策树。

5.4.1 问句解析

问句解析依次识别设备、区域和品牌：

（1）设备：把问句归一化后，按长度从长到短与设备词典（来自图谱设备节点和原始记录的工位字段）匹配，长编号优先；匹配位置后紧跟数字时视为未命中，避免“EM21-B1”误命中“EM21-B12”。

（2）区域和品牌：在去掉设备号后的剩余文本中，按整词匹配区域名和设备台账中的品牌。

（3）剩余文本：去掉设备号、区域、品牌以及“是什么原因”“怎么办”等疑问尾巴和标点后，剩下的部分作为故障描述，用于后续打分。

识别出设备时，只在该设备的记录中检索；未识别设备但识别出品牌或区域时，在对应设备集合中检索。该步骤纯属字符串处理，实测耗时不到 1 毫秒（第 6 章）。

5.4.2 混合打分

对候选记录分别计算字面分和向量分：

（1）字面分。中文按相邻两个字切成二元字组，英文和数字按整词切分，不需要分词词表。定义覆盖度为集合 A 中有多少（按 IDF 加权）出现在集合 B 中：

$$\mathrm{cov}(A,B)=\frac{\sum_{g\in A\cap B}\mathrm{idf}(g)}{\sum_{g\in A}\mathrm{idf}(g)} \tag{5-1}$$

其中 IDF 用全部记录统计，采用平滑形式 idf(g)=ln((N+1)/(n_g+1))+1。设问句故障描述的字组集合为 Q，记录的“部件+故障名+原始现象”字组集合为 D，记录的简短名称（部件+故障名）字组集合为 S，字面分为

$$s_{\mathrm{lex}}=0.7\mathrm{cov}(Q,D)+0.3\mathrm{cov}(S,Q) \tag{5-2}$$

前一项衡量问句被记录覆盖了多少（召回），后一项衡量记录的简短名称被问句覆盖了多少（精度），防止只因记录文本长、碰巧包含问句中的字而得高分。

（2）向量分。用 BGE-M3 编码问句，在 Qdrant 中按设备过滤后取最相近的记录，余弦相似度为 v。由于 BGE-M3 在本数据上的余弦值大多落在 0.45～0.85 之间，先线性映射到[0,1]：

$$s_{\mathrm{vec}}=min\left(1,max\left(0,\frac{v-0.45}{0.4}\right)\right) \tag{5-3}$$

（3）综合分按分数加权融合（第 2.3.1 节）计算：

$$s=0.65s_{\mathrm{lex}}+0.35s_{\mathrm{vec}} \tag{5-4}$$

向量服务不可用时只用字面分。按综合分降序排序后，以 max(0.25, 0.4×最高分) 为阈值截断，取前 15 条作为相似记录；另外从全部候选中单独取前 40 条带图谱链路的记录作为图谱候选，因为只有 2023 年的记录有链路，若只从前 15 条中挑选，会被 2024、2025 年的记录挤掉。权重 0.65 与其他融合方式的对比见

第 6 章。

（4）可选精排。设置中打开“精排”后，用 bge-reranker 对召回结果重排序。实验发现问句与记录的输入格式不对齐会使精排降低命中率，因此精排的输入改为“问句去掉设备号”对“部件+故障名+现象”；由于对齐后精排也没有提升命中率，且每次增加约 5 秒，系统默认关闭精排（第 6 章）。

（5）实验屏蔽。做留一法实验时，通过环境变量 EVAL_EXCLUDE_SRC 指定测试记录的来源编号，检索时把该记录及其图谱链路从候选中剔除，防止大模型“看着答案作答”。该开关只在实验脚本中设置，正常使用时不生效。

5.4.3 图谱子图的选取

图谱候选中并非每条链路都适合画出来：原始描述中顺带提到的部件（如“修刀库门时发现安全门松动”中的安全门）会让图谱变得杂乱。因此只画节点名与问句“整词对得上”的链路：问句点名了部件时，只画该部件的链路；否则画故障名对得上的链路。“故障”“报警”这类泛化词单独出现时不算命中，但问句去掉这些词后的主体（如“Y 轴故障”中的“Y 轴”）出现在节点名中也算命中。选出的链路组装成节点和边：同一对节点之间来自不同记录的平行边合并为一条显示，边上带上它们的来源编号列表（source_rows），供前端高亮使用。

5.5 排查决策树生成

5.5.1 决策树的数据结构

无论由大模型还是规则生成，决策树都用同一种 JSON 结构描述（称为 DSL），主要字段见表 5-1。前端把它转换成 Mermaid 流程图描述后渲染。

表 5-1 决策树 DSL 的主要字段

字段	含义
problem	故障问题（≤20 字）
likely_cause	最可能的原因
mode	single：单一路径；by_cause：按原因分支
steps	通用前置检查步骤（check / action / why / evidence）

字段	含义
branches	原因分支：cause，steps[{do, why}]，evidence（证据编号）
cause_summary	原因—措施对照表
tip / verify	经验提示 / 处理后的验证条件
generator / fallback_reason	生成方式（llm / rule）/ 改用规则的原因

5.5.2 大模型生成

（1）证据组织。相似记录取前 6 条，每条写成一行“[编号] 工位、问题、原因、措施、日期、相似度、成本等级”，成本等级由 5.5.4 节的规则给出，让大模型的排序有据可依。图谱证据最初按类别列出（“原因：A|B|C；措施：X|Y|Z”），这种写法丢失了“哪个原因对应哪个措施”；改为每行一条“部件 → 故障 → 原因 → 措施”链路，相同链路合并并标注出现次数。

（2）提示词。提示词要求：只能使用证据中出现的部件和处理动作；每个分支必须注明依据的证据编号；证据对不上问句时如实说明，不要硬凑分支；按“与问题的吻合程度 + 出现次数”挑选原因；只输出 JSON。旧版提示词中有一个写死的编码器示例，模型常照抄，新版删除了示例。新旧提示词在 20 条开发集上的对比见第 6 章 6.x 节，新版措施命中率提高 15 个百分点，无依据词比例降低 10.6 个百分点。

（3）调用与解析。通过 OpenAI 兼容接口调用，温度为 0，超时 45 秒。返回内容先去掉 Markdown 代码块标记，再提取第一个 JSON 对象解析；各字段按长度截断，分支最多 4 个，每个分支最多 3 步。解析失败、或解析后既无步骤也无分支，视为输出无效。

（4）分支重排。大模型给出的分支顺序不一定稳定，系统不直接采用，而是用 5.5.4 节的规则为每个分支计算成本等级后按等级升序重排（同级保持大模型的原顺序），原因—措施对照表随之同序。这样，“先查省钱省时的原因”由确定的规则保证，同一输入的分支顺序可复现。

5.5.3 规则生成与降级

规则决策树不调用大模型，在检索阶段就已生成，作为备用：

（1）把检索到的记录按原因分组，图谱记录用原因节点名，没有链路的记录取原始原因文本的第一句（不超过 16 字）；“未找到原因”“交接”等无效原因跳过。

（2）每组累计相关度之和 w（各记录综合分之和），即“相关度×出现次数”，取 w 最大的前 4 组作为分支。

（3）每个分支取该原因下出现最多的两种处理措施作为处理步骤，并注明“历史 N 次·成本/耗时等级 L”。

（4）分支按成本等级升序排列，同级按出现次数降序。

决策树生成入口按以下顺序决定生成方式：配置为“只用规则”或没有检索结果时，直接采用规则决策树；否则探测大模型服务——对服务地址建立 TCP 连接，3 秒内连通视为在线，结果缓存 30 秒，避免每次提问都探测；“模拟断网”开关打开时直接视为离线。在线时调用大模型，出现网络异常、超时或输出无效时，改用规则决策树，并在结果中记录原因（如“大模型输出无效（ValueError: …）”），前端悬停在“规则生成”标识上即可看到。

5.5.4 成本等级的确定

成本等级用关键词规则从处理措施和原因文本中推断，分成本和耗时两个维度，各分 5 级（表 5-2）。一段文本命中多个等级时取最高级，因为一条记录最终若要“更换电机”，其成本应按更换计算，而不是按其中的“检查”步骤计算；没有命中任何关键词时取 2 级。综合等级为两者平均后四舍五入。为避免型号插在动词中间导致漏配（“更换 C32 编码器”不含“更换编码器”），评分前先去掉型号和工位编号。

表 5-2 成本与耗时等级（部分关键词）

等级	成本关键词	耗时关键词
1	目视、检查、紧固、复位、重启、清洁、调整参数	目视、确认、复位、重启、紧固、插紧

等级	成本关键词	耗时关键词
2	焊接、接线、润滑、加油	清洁、清理、吹扫、接线、润滑、调整
3	更换传感器/感应器/编码器、打磨	焊接、更换小件、打磨
4	更换轴承/油封/主轴、拆装	更换轴承/油封、拆装
5	更换电机/齿轮箱/减速机/总成、整机更换	更换总成/电机、等备件

5.6 前端实现

5.6.1 分阶段展示

前端用 fetch 读取 SSE 流，按帧的 type 字段分别更新三个区域：收到 diagnosis_kg 即渲染知识图谱，收到 diagnosis_records 即渲染相似记录表，收到 diagnosis_flowchart 再渲染决策树，决策树生成期间显示等待提示。这样用户在约 2 秒后就能看到图谱和记录（第 6 章 E4），不必等待大模型。

5.6.2 知识图谱的单条链路高亮

图谱用 Cytoscape.js 和 fcose 力导向布局绘制，节点按六类实体着色。由于节点共享，同一个部件节点可能连着多条记录的链路。点击某个部件时，以该节点为起点做广度优先遍历，只沿边上 source_row 列表与当前链路有交集的边前进，从而只点亮属于同一条维修记录的完整链路，其余元素淡出；点击空白处恢复。选中节点时显示其关联的来源记录编号，有编辑权限的用户还可以编辑节点或把它合并到另一个节点。

5.6.3 决策树渲染

前端把决策树 DSL 转换为 Mermaid 流程图描述：问题为起点，通用检查步骤依次相连，进入原因分支后每个分支一列，列出处理步骤，最后汇合到验证条件。标题栏显示“大模型生成”或“规则生成”标识。流程图可以缩放、全屏查看、收藏到流程图库，诊断结果可导出为 PDF、Word 或图片。

【图 5-x 诊断页面截图（用脱敏数据截取：问句、图谱、记录表、决策树各一张）】

5.7 本章小结

本章介绍了系统关键技术的实现：用“字段定义 + 难例规则 + 输出约束”的提示词抽取六元组，经规则校验、专家复核和实体规范化后导入 Neo4j，用关系上累积的来源编号实现溯源；统一检索在一次打分中综合二元字组字面覆盖度和向量相似度，同一批结果供图谱、记录表和决策树共用；决策树优先由大模型融合两路证据生成，分支顺序由确定的成本规则重排，大模型不可用时自动改用规则决策树；前端按阶段展示结果，并支持单条链路高亮。

第 6 章 测试与实验

本章先对系统主要功能进行测试，再通过四组实验评估系统：图谱构建质量（E1）、检索效果（E2）、排查决策树生成的消融实验（E3）和系统性能（E4）。

6.1 实验环境与数据

实验在一台笔记本电脑上进行：Windows 操作系统，CPU 运行，内存 7.3 GB；Neo4j、Qdrant、BGE-M3 嵌入模型和后端服务都部署在本机；大模型通过企业内部的模型网关调用（OpenAI 兼容接口，温度 0）。【待补：CPU 型号、Neo4j/Qdrant 版本、大模型名称】

实验数据为某机加工车间 2023—2025 年的维修记录共 15319 条，知识图谱由 2023 年经专家复核的 5454 条六元组构建（表 4-3）。检索实验和消融实验都在 2023 年有图谱链路的记录中抽样，采用留一法：以一条记录的“设备+原始故障现象描述”作为问句，检索和生成时把这条记录本身及其图谱链路屏蔽掉，看系统能否依靠其他历史记录给出正确结果，相当于“来了一个新故障，系统能否找到以前同类的处理经验”。各实验的随机种子固定，结果可复现。

6.2 功能测试

功能测试分接口层和界面层两部分。接口层用测试脚本（tests/func_test.py）对运行中的后端逐项调用并自动判定，图谱编辑类测试使用临时节点，测试后删除，不影响业务数据；界面层在浏览器中人工操作。接口层结果见表 6-1，21 项全部通过。

表 6-1 功能测试结果（接口层）

测试项	操作	预期结果	实际结果	结论
带设备号诊断	提问“设备号+刀库门打不开”	依次返回图谱、记录、决策树	图谱 22 个节点、记录 15 条、决策树 4 个分支，大模型生成	通过
分阶段返回	记录各帧到达时间	图谱、记录早于决策树	图谱、记录约 1 s，决策树约 40 s	通过
相似记录溯源	检查记录的来源字段	每条有来源文件和行号	15/15 条	通过

测试项	操作	预期结果	实际结果	结论
图谱边溯源	检查图谱边的来源编号	每条边带来源记录编号	23/23 条	通过
不带设备号诊断	提问“刀库门打不开”	在全部记录中检索并给出结果	记录 15 条、决策树 4 个分支	通过
英文诊断	英文提问	给出与中文提问一致的结果	记录 15 条、决策树 4 个分支	通过
追问	同一会话追问	基于本会话结果回答，不重新检索	返回文字回答，未推送图谱帧	通过
会话保存	查询会话消息	保存问句与回答	4 条消息	通过
模拟断网	打开“模拟断网”后提问	规则生成并注明原因	规则生成（无网络），决策树立即返回	通过
只用规则	生成方式选“只用规则”	规则生成	规则生成	通过
关闭知识图谱	关闭“知识图谱”开关	不返回图谱，仍有记录	图谱 0 个节点、记录 15 条	通过
关闭向量库	关闭“向量库”开关	不返回记录，仍有图谱	图谱 24 个节点、记录 0 条	通过
新建节点	新建临时部件节点	创建成功	成功，返回节点编号	通过
修改节点	修改节点名称	修改成功	成功	通过
回滚修改	按变更日志回滚	名称恢复	成功，按原名可搜到	通过
删除节点	删除临时节点	删除成功	成功	通过
图谱统计	查询图谱统计	返回节点、关系数	7075 个节点、28815 条关系	通过
提示词版本历史	查询提示词历史版本	返回版本列表	成功	通过

测试项	操作	预期结果	实际结果	结论
流程图库	查询收藏的流程图	返回列表	成功	通过

注：服务可用性检查和“回滚后名称恢复”两项未单列。测试过程中有一次首帧耗时达 22 s，原因是检索索引缓存每 120 s 过期一次、过期后的第一次提问要同步重建索引（约 10–20 s）；已改为过期后先用旧索引、后台重建（见 6.6 节）。

界面层测试项（人工）：【待补：图谱点击部件高亮单条链路、记录表点击溯源弹窗、决策树“大模型生成/规则生成”标识与悬停原因、只读角色看不到编辑按钮、流程图收藏与导出、中英文切换、数据导入。】

6.3 实验一：图谱构建质量（E1）

6.3.1 实验方法

从专家复核后的 2023 年六元组中，按设备分层随机抽取 200 条（每台设备的抽样数与其记录数成比例，随机种子 20261109），对部件、故障现象、原因、处理措施四个字段逐一判定是否正确，统计字段准确率和整条准确率（四个字段都正确才算整条正确），并给出 Wilson 95%置信区间。区域和设备直接取自记录字段，不在抽检范围内。判定标准见附录 A，要点是：部件须为具体发生故障的部件，写成动作、维修过程或设备整体算错；原因须能从对策推断，推断不出却硬写、或与现象重复的算错，原始记录确实看不出原因时写“未知”算对；措施须为最终有效的处理动作，只写中间尝试算错。

由于逐条判定工作量大，原计划由作者判定前 50 条，其余 150 条用大模型按同一标准辅助判定，并先检验两者的一致性。结果显示，大模型与作者判定的一致性很低（49 条，四个字段的 Cohen kappa 为 0.12～0.42，一致率 59%～74%），不能代替人工，因此本节只报告作者人工判定的结果。

6.3.2 实验结果

作者判定了前 50 条，其中 1 条原因字段拿不准未填，有效 49 条，结果见表 6-2。

表 6-2 六元组抽取准确率（人工判定，49 条）

字段	准确率	95%置信区间
部件	65.3%（32/49）	[51.3%, 77.1%]
故障现象	71.4%（35/49）	[57.6%, 82.2%]
故障原因	55.1%（27/49）	[41.3%, 68.1%]
处理措施	65.3%（32/49）	[51.3%, 77.1%]
整条	32.7%（16/49）	[21.2%, 46.6%]

整条准确率只有 32.7%，远低于开题报告中“不低于 90%”的目标。由于样本只有 49 条，置信区间较宽，但即使取上限（46.6%）也与目标差距很大，结论是确定的：当前的抽取结果作为“逐条精确的知识”质量不足。

6.3.3 结果分析

逐条查看判错的记录，错误主要有以下几类。

（1）原因字段硬补失效模式（原因准确率最低，55.1%）。原始记录常常只写了处理动作，例如“更换气杯一个”“清理调整感应器”，模型却补成“气杯密封老化”“感应器堵塞”。这类“原因”看起来合理，但原始记录中没有依据。原因在于抽取提示词要求原因写成“部件＋失效模式”，并要求尽量不要填“未知”，模型于是倾向于补出一个最常见的失效模式。反过来，也有 2 条记录明明能从对策看出原因，模型却填了“未知”。

（2）部件写成现象或写得过粗。例如把“漏水”“漏气”填为部件，或者故障实际出在某个感应器、线束上，却只写了“刀塔”“Y 轴”这样的上级部件。

（3）交接记录只抽到中间步骤。维修记录中有一类是交班时的“交接”记录，写的是尚未完成的排查过程（如“现准备拆钣金检查电机（交接）”）。抽检的 49 条中有 9 条是交接记录，整条只对了 2 条（非交接记录为 14/40）。这类记录本身没有最终处理结果，抽出的措施往往只是“检查电机”这类中间步骤。

（4）原始记录本身信息不足。有的记录故障现象和对策写的是同一句话，如“自行维修旋转接头”“更换 AGV 后轮”，看不出故障现象和原因；有的对策只是“修改参数 32200”。这几条记录四个字段很难全部正确，3 条“现象与对策相同”的记录整条全错。

需要说明的是：①抽检的判定标准较严，整条正确要求四个字段同时正确；②故障现象（71.4%）和处理措施（65.3%）这两个与排查直接相关的字段好于原因字段；③系统在诊断时并不只依赖图谱：E3 表明，大模型同时拿到相似原始记录时，效果主要来自原始记录，图谱的增益有限（6.5 节），这与图谱质量不高的结论一致。

改进方向是：抽取时允许并鼓励在原因推断不出时填“未知”，原因只写原文能推出的内容；对交接记录与后续完成记录做合并后再抽取；对部件字段增加“不得为现象词”的规则校验。这些作为第 7 章的展望。

6.4 实验二：检索效果（E2）

6.4.1 知识图谱子图检索（E2a）

从 2023 年有图谱链路的记录中随机抽取 300 条（随机种子 7），对比旧版精确匹配方法与本文方法。相关链路定义为同一设备、同一部件的其他链路。评价指标：空结果率（一条链路都没查到的比例）、含同类率（结果中至少有一条相关链路的比例）、链路精度（结果中相关链路所占比例，空结果记 0）。结果见表 6-3。

表 6-3 知识图谱子图检索对比（300 条）

方案	空结果率	含同类率	链路精度	平均链路数	耗时中位数
A 精确匹配（旧版）	50.3%	40.0%	34.7%	2.8	0.02 s
B1 词典识别 + 字面打分	8.0%	82.7%	74.7%	4.3	0.01 s
B2 词典识别 + 字面/向量混合（本文）	7.3%	83.3%	75.3%	4.4	0.77 s

旧版方法要求问句中的词与节点名完全一致，口语化的问句有一半查不到任何链路。改用图谱实体词典识别设备、按字面覆盖度打分后，空结果率从 50.3% 降到 8.0%，含同类率从 40.0%提高到 82.7%。加入向量相似度后各项略有提高，但耗时增加约 0.76 秒，主要用于问句编码和向量检索。

6.4.2 相似记录检索（E2b）

采用同样的留一法，相关记录定义为同一设备、同一部件的其他历史记录。评价指标为 Hit@1、Hit@5（前 1、前 5 条中至少有一条相关记录的比例）、MRR（第一条相关记录名次倒数的平均值）和 P@5（前 5 条中相关记录的比例）。抽样 300 条，去掉没有任何同类记录的样本后有效 285 条。8 种打分方案的结果见表 6-4。

表 6-4 相似记录检索方案对比（285 条）

方案	Hit@1	Hit@5	MRR	P@5
S1 纯字面	83.2%	93.7%	0.875	61.3%
S2 纯向量（BGE-M3）	79.3%	94.4%	0.861	58.7%
S3 字面 0.65 + 向量 0.35（本文）	83.9%	95.8%	0.888	62.4%
S4 S3 + 原因/措施加入检索文本	83.5%	95.1%	0.885	61.5%
S5 字面 0.4 + 向量 0.6	83.2%	95.8%	0.884	61.7%
S6 RRF 融合	83.2%	95.8%	0.883	61.8%
S7 S3 + 部件命中加分	84.9%	95.8%	0.894	63.5%
S8 RRF + 部件命中加分	86.0%	95.8%	0.898	63.9%

由于部分设备的同部件历史记录不足 5 条，P@5 的理论上限为 70.9%。从表中可以看出：①字面检索明显好于单纯的向量检索（Hit@1 高 3.9 个百分点），说明在设备编号、部件名称密集的维修短文本上，精确的字面匹配很重要；②字面与向量加权融合（S3）在四项指标上都不低于单独使用其中一路，Hit@5 达到 95.8%；③加入部件命中加分的 S7、S8 的 Hit@1 比 S3 高 1.0～2.1 个百分点，但在 285 条样本上差异不显著。系统采用实现简单、不需要额外排名计算的 S3。把原因和措施加入检索文本（S4）没有带来提升，因为用户提问时只描述现象。

用精确 McNemar 检验对 S3 与其他方案做配对比较：Hit@1 上，S3 与纯向量（S2）的差异显著（S3 独中 16 条、S2 独中 3 条，p=0.004）；与 S8 的差异不显著（S8 独中 12 条、S3 独中 6 条，p=0.238），与其余方案的 p 值均大于 0.25；Hit@5 上各方案与 S3 都没有显著差异。

6.4.3 交叉编码器精排（E2b 续）

在 S3 召回结果的基础上，取前 30 条用 bge-reranker 精排，与只召回的结果比较（表 6-5）。

表 6-5 精排对比（285 条）

方案	Hit@1	Hit@5	MRR	P@5	精排耗时（中位数）
R0 只召回	82.1%	95.4%	0.877	61.5%	—
R1 精排，原输入格式	68.4%	91.6%	0.785	52.4%	14.1 s
R1 精排，对齐输入格式	80.7%	93.7%	0.866	59.8%	5.3 s

注：本表 R0 与表 6-4 的 S3 公式相同，召回结果为 2026 年 11 月 9 日的缓存，数据经实体规范化前后有差异，因此数值略有不同；表内各行使用同一份召回结果，彼此可比。

初次接入精排时，Hit@1 反而从 82.1%降到 68.4%。分析精排把原本正确的第 1 名换掉的 53 个样例发现，换上来的多是原因、措施文字较长的记录：问句只有现象，记录文本却拼接了现象、原因、措施和设备号，交叉编码器偏向信息量多的长文本。把输入格式对齐——问句去掉设备号，记录只保留部件、故障名和现象——之后，精排与召回的差距从-13.7 个百分点缩小到-1.4 个百分点，耗时也降到约三分之一。再把精排与召回结果做 RRF 融合，Hit@1 为 82.3%～82.7%，与只召回基本持平（266 条，差异在抽样误差内）。可见，在本数据上，召回阶段的字面+向量混合打分已经比较准确，通用的交叉编码器没有带来额外收益，反而每次增加 5 秒以上。因此系统默认关闭精排，保留为可选功能。

6.5 实验三：排查决策树生成消融实验（E3）

6.5.1 实验设计

从 2023 年有图谱链路的记录中随机抽取 100 条作为测试集（随机种子 20261111），排除提示词开发集的 20 条，以及真实原因为“未找到原因”的记录。设置五组对照（表 1，开题报告）：G0 为规则决策树；G1 只给大模型问句；G2 给问句和相似记录；G3 给问句和知识图谱链路；G4 给问句、图谱链路和相似记录（本文系统）。各组都调用系统真实的决策树生成入口（含规则兜底），只改

变提供的证据，精排关闭。

自动评价指标：

（1）根因命中率：决策树任一分支的“原因+处置”文本与该记录真实原因的字面覆盖度不低于 0.5（二元字组，IDF 加权），或向量余弦相似度不低于 0.80，即算命中。

（2）措施命中率：判定方法同上，比对真实处理措施。

（3）无依据词比例：决策树中出现、但在完整检索证据中找不到的实词所占比例，用来近似衡量“幻觉”。

（4）大模型生成率：最终结果由大模型生成（而不是改用规则）的比例。

两个阈值在 20 条开发集上校准后固定，测试集上不再调整。

6.5.2 实验结果

表 6-6 消融实验结果（100 条）

组别	根因命中 [95%CI]	措施命中 [95%CI]	无依据词	大模型生成	平均分支	平均耗时
G0 规则（图谱+记录）	36% [27,46]	33% [25,43]	9.5%	0%	3.5	10.2 s
G1 大模型（无证据）	2% [1,7]	4% [2,10]	43.2%	58%	0.0	17.5 s
G2 大模型+相似记录	41% [32,51]	37% [28,47]	34.6%	97%	2.6	39.0 s
G3 大模型+图谱	29% [21,39]	19% [13,28]	20.2%	78%	1.8	22.0 s
G4 大模型+图谱+记录（本文）	46% [37,56]	34% [25,44]	34.2%	96%	2.8	43.9 s

注：耗时包含检索与决策树生成，仅供组间粗略比较，系统性能以 6.6 节 E4 为准。G0 的耗时几乎全部在检索阶段（中位数 6.8 s，决策树生成接近 0），明显高于其他组同一批问句的检索耗时（G4 中位数 1.5 s）和 E4 的测量值（2.38 s）。

原因是检索索引缓存每 120 s 过期，过期后的第一次检索要从 Neo4j 和原始记录重建索引（实测 13–18 s）；实验中每条问句依次跑 G0–G4，大模型组每次约 40 s，轮到下一条问句的 G0 时缓存通常已过期，重建开销几乎都落在 G0 上（检索超过 4 s 的问句，G0 有 62 条，其他组为 5–25 条）。这一开销与组别无关，不影响命中率等指标。发现该问题后，系统改为缓存过期时先返回旧索引、在后台重建。

由于五组在同一批问句上测试，用 McNemar 配对检验比较 G4 与其他各组（表 6-7）。

表 6-7 G4 与其他组的配对检验

对比	根因：G4 独中/对方独中，p 值	措施：G4 独中/对方独中，p 值
G4 vs G0	16 / 6，p=0.052	8 / 7，p=1.000
G4 vs G1	44 / 0，p<0.001	31 / 1，p<0.001
G4 vs G2	8 / 3，p=0.227	2 / 5，p=0.453
G4 vs G3	20 / 3，p<0.001	16 / 1，p<0.001

6.5.3 结果分析

（1）检索证据是大模型给出正确结果的前提。没有证据时（G1），大模型的根因命中率只有 2%，而且有 42%的问句连有效的决策树都生成不出来（输出为空，被改用规则）。这说明通用大模型不了解该车间设备的具体情况，直接用于诊断不可行。

（2）在只有图谱证据的基础上加入相似记录（G3→G4），根因命中率从 29%提高到 46%，措施命中率从 19%提高到 34%，差异显著。图谱链路数量少（平均每题只有几条），单独使用时证据面窄；相似记录提供了更多原始细节。

（3）在相似记录的基础上加入图谱（G2→G4），根因命中率从 41%提高到 46%，但差异不显著（p=0.227），措施命中率没有提高。也就是说，在本测试集上，图谱对大模型的增益主要体现在原因定位上，幅度有限。可能的原因是：图谱链路本身就由同一批维修记录抽取而来，与相似记录包含的信息大量重叠；而经过规范化的图谱节点名较短，措施信息不如原始记录具体。这一结果说明，知识图谱的价值更多在于结构化展示、溯源和对原因的归纳，而不是单纯提高命

中率。

（4）与规则方法（G0）相比，G4 的根因命中率高 10 个百分点，接近显著（p=0.052），措施命中率相当。规则方法的无依据词比例最低（9.5%），因为它直接摘抄原始记录的原因和措施，不会产生新内容；大模型各组的无依据词比例为 20%～35%，其中包含“测试确认”“观察是否卡死”等为执行步骤补充的通用说法，并不都是编造，这一指标只能作为参考，需要结合人工评价。

（5）大模型生成决策树每次约 40 秒，规则方法几乎不耗时（E4）。在大模型不可用时改用规则决策树，根因命中率从 46%降到 36%，结果仍然可用。

6.5.4 人工评价

自动指标只做字面和向量匹配，同义的不同说法会被判错，也无法评价排查顺序是否合理，因此另做人工评价。评分标准为 1～5 分（评分标准见附录 B）：5 分为原因和处理都对得上该故障、可直接照做；3 分为部分有用但混有明显无关原因或措施笼统；1 分为完全不相关、有误导，或内容为空。

由于 100 条×5 组共 500 个决策树，作者从中随机抽取 10 条问句，把五组生成的决策树打乱顺序、隐去组别，站在现场维修工程师的角度盲评，共 50 个；另用大模型（温度 0）按同一标准对全部 500 个决策树评分。大模型与作者在 50 个决策树上的一致性为：加权 kappa 0.81，Spearman 相关系数 0.77，完全一致 60%，相差不超过 1 分 94%。加权 kappa 在 0.8 以上，一般认为一致性很好，因此以大模型评分作为 100 条的人工评价结果，作者盲评作为校验。结果见表 6-8。

表 6-8 决策树质量评分（1～5 分）

组别	大模型评分：平均分	≥4 分比例	作者盲评：平均分（10 条）	≥4 分比例
G0 规则（图谱+记录）	3.09	38%	2.70	20%
G1 大模型（无证据）	1.60	0%	1.90	10%
G2 大模型+相似记录	3.74	65%	3.30	50%
G3 大模型+图谱	2.84	32%	2.90	40%
G4 大模型+图谱+记录（本文）	3.86	68%	3.80	80%

注：决策树内容为空（大模型和规则都没有给出结果）的按评分标准记 1 分，作者盲评的 50 个中有 7 个。

用配对符号检验比较 G4 与其他各组在同一问句上的分数（大模型评分，100 条）：G4 对 G0 为 62 胜 2 负 36 平，对 G1 为 93 胜 1 负 6 平，对 G3 为 61 胜 5 负 34 平，p 均小于 0.001；对 G2 为 20 胜 9 负 71 平，p=0.061，不显著。

人工评价与自动指标的结论一致，并且更清楚：①G4 得分最高，作者盲评中 80%的决策树达到“基本可用”（4 分）以上；②没有证据的 G1 几乎不可用；③大模型＋检索证据（G2、G4）明显好于规则方法 G0，这一点在自动指标上只是接近显著（根因 p=0.052），在质量评分上显著（p<0.001），说明大模型的价值不只在“找对原因”，还在于把多条证据组织成步骤具体、顺序合理的排查流程；④加入图谱（G2→G4）仍有小幅提高，但不显著，与 6.5.3 节的结论相同。

6.5.5 提示词与证据格式的选型

E3 之前，在 20 条开发集（随机种子 20261110，与测试集不重叠）上比较了三种方案（表 6-9）。

表 6-9 提示词与证据格式对比（开发集 20 条）

方案	格式合法	根因命中	措施命中	无依据词	平均分支
A 旧提示词 + 图谱按类别列节点	95%	37%	32%	35.3%	3.4
C 旧提示词 + 图谱按链路	100%	40%	30%	42.2%	3.5
B 新提示词 + 图谱按链路（本文）	95%	42%	47%	24.7%	3.2

新提示词配合按链路组织的图谱证据，措施命中率提高 15 个百分点，无依据词比例降低 10.6 个百分点；只改证据格式、不改提示词（C）效果不明显。由于开发集只有 20 条，差异不显著，这一结果只作为选型依据，最终效果以测试集为准。

6.6 实验四：系统性能（E4）

取 E3 测试集的前 30 条问句（带设备号），在默认配置下（图谱和相似记录

开启、精排关闭）分阶段计时，结果见表 6-10。测试脚本直接调用后端的检索与生成函数，测试时只运行 Neo4j 和 Qdrant，关闭后端服务和浏览器，避免两份向量模型同时占用内存。在这台 7.3 GB 内存的笔记本上，机器负载对检索耗时影响明显，前期在后端同时运行、空闲内存不足 1.2 GB 时测得的完整检索中位数达 3～5 秒，这些结果未采用。

表 6-10 诊断各阶段耗时（30 条）

阶段	中位数	95 分位	均值
问句解析（设备/区域/品牌识别）	0.2 ms	1.0 ms	0.4 ms
向量检索（BGE-M3 编码 + Qdrant）	1.23 s	2.43 s	1.49 s
完整检索（含解析、向量、字面打分、图谱链路）	2.38 s	5.39 s	2.77 s
决策树生成：大模型（30 次均成功）	41.2 s	60.5 s	40.2 s
决策树生成：规则	0.7 ms	5.0 ms	1.5 ms
端到端（完整检索 + 大模型决策树）	44.5 s	62.3 s	42.9 s

问句不带设备号、需要检索全部 1.5 万条记录时（5 条），完整检索的中位数为 1.20 秒，最长 1.99 秒。

可以看出：①检索阶段的中位数为 2.38 秒，略高于第 3 章提出的“2 秒内返回”，未完全达到要求。其中约一半是向量检索（主要是 BGE-M3 在 CPU 上编码问句），其余为字面打分和图谱链路查询。测试机是无独立显卡、7.3 GB 内存的笔记本，部署到有 GPU 或更大内存的服务器上向量编码应会加快，但本文未在服务器上实测。由于采用分阶段推送，用户约 2 秒后就能看到图谱和相似记录，不必等待大模型。②端到端时间的约 95%花在大模型生成上，取决于外部模型服务，95 分位约 1 分钟，符合“一般 1 分钟内完成”的要求。③无网络时规则决策树只需毫秒级，整个诊断约 2 秒完成。④索引缓存改为过期后后台重建、有效期延长到 1 小时后，测量中没有再出现十几秒的检索卡顿，完整检索的 95 分位为 5.39 秒。

6.7 本章小结

本章对系统进行了功能测试和四组实验。图谱检索方面，词典识别加混合打分使空结果率从 50.3%降到 7.3%；相似记录检索方面，字面与向量加权融合的

Hit@5 达到 95.8%，通用交叉编码器精排在本数据上没有带来提升，系统默认关闭；消融实验表明，检索证据是大模型给出正确结果的前提，图谱和相似记录合用时根因命中率最高（46%），但图谱相对于只用相似记录的增益不显著；性能方面，在笔记本 CPU 上检索中位数约 2.4 秒，略高于 2 秒的设计目标，主要耗时在大模型生成（约 40 秒），无网络时整个诊断约 2 秒完成。人工评价方面，G4 的决策树质量评分最高（平均 3.86 分，68%达到 4 分以上），显著高于规则方法和无证据的大模型；图谱构建质量的人工抽检显示整条准确率只有 32.7%，主要问题是原因字段硬补失效模式和交接记录信息不完整，未达到开题目标，是后续需要改进的重点。

第 7 章 总结与展望

7.1 工作总结

本文针对机加工车间设备维修经验难以复用的问题，设计并实现了一个基于知识图谱增强大模型的产线故障诊断系统，主要完成了以下工作：

（1）构建了机加工车间设备故障知识图谱。设计了区域、设备、部件、故障现象、原因、处理措施六类实体和五类关系的故障本体；采用大模型抽取、规则校验、专家复核和实体规范化相结合的方式，从 2023 年 5455 条维修记录中构建了包含 6993 个实体节点的知识图谱，不需要标注训练数据；关系上保存来源记录编号，使每条知识都能追溯到原始记录。人工抽检 49 条六元组，故障现象、处理措施准确率分别为 71.4%和 65.3%，原因字段只有 55.1%，整条准确率 32.7%，未达到开题设定的目标。

（2）实现了统一检索方法。用设备词典识别问句中的设备，用二元字组的 IDF 加权覆盖度计算字面相关度，与 BGE-M3 向量相似度按 0.65:0.35 加权，一次检索同时服务图谱、记录表和决策树三处。图谱子图检索的空结果率从旧版的 50.3%降到 7.3%，相似记录检索的 Hit@1 为 83.9%、Hit@5 为 95.8%，字面与向量融合在四项指标上都不低于单独使用其中一路。

（3）实现了知识图谱增强的排查决策树生成及降级机制。把图谱链路和相似记录组织成带编号的证据，约束大模型只使用证据中的部件和动作，输出结构化的决策树；分支顺序由确定的成本规则重排，保证“先易后难”可复现；无网络、超时或输出不合格时自动改用规则决策树，保证始终有结果。

（4）通过消融实验分析了两路证据的作用。在 100 条留一法测试问句上，没有检索证据时大模型的根因命中率只有 2%；图谱与相似记录合用时根因命中率为 46%，显著高于只用图谱（29%），高于规则方法（36%，p=0.052）；但相对于只用相似记录（41%）的提升不显著。人工质量评分（经作者盲评校验，加权 kappa 0.81）给出相同的结论：系统方案平均 3.86 分，规则方法 3.09 分，无证据的大模型 1.60 分。这说明在本数据上，检索证据是大模型可用的前提，知识图谱的价值更多体现在结构化展示、溯源和对原因的归纳上。

（5）完成了可现场使用的系统。在笔记本 CPU 上检索阶段约 2 秒完成（中位数 2.38 秒，略高于 2 秒的设计目标），结果分阶段返回；大模型生成决策树约 40 秒；无网络时整个诊断约 2 秒完成。

7.2 不足与展望

（1）知识抽取质量有待提高。人工抽检的整条准确率只有 32.7%，主要问题有三类：一是抽取提示词要求原因写成“部件＋失效模式”并尽量不填“未知”，原始记录推不出具体原因时，模型倾向于补出看似合理的失效模式；二是部件字段有时写成现象（如“漏水”）或过于笼统的上级部件；三是交班“交接”记录只有中间排查步骤，没有最终结果。后续可以允许模型在推不出时填写“未知”、原因只写原文能推出的内容；把交接记录与后续完成记录合并后再抽取；对部件字段增加规则校验；并以人工标注的小样本评估每次改进的效果。此外，大模型在逐条判定抽取结果时与人工一致性较低（kappa 0.12～0.42），抽取质量评估目前仍需依靠人工。

（2）知识图谱对大模型的增益有限。图谱由同一批维修记录抽取而来，与相似记录信息重叠较多，规范化后的节点名也较为简短。后续可以在图谱中加入设备结构（部件层级）、故障模式分类等记录中没有的知识，让图谱提供相似记录之外的信息；也可以利用图谱做多跳推理，例如从同类部件的其他设备上查找经验。

（3）评价方法有待完善。根因和措施命中率基于字面和向量相似度自动判定，口径偏严，同义的不同说法可能被判为未命中；人工评价的样本量也有限。后续可以邀请多名维修工程师参与评价，并在现场试用中统计实际排查时间的变化。

（4）图谱目前只覆盖 2023 年的记录。2024、2025 年的记录已向量化、可用于相似记录检索，但尚未抽取进图谱。后续可以用本文的流程把新记录增量补充进图谱，并研究自动发现新增同义实体的方法，减少人工复核工作量。

（5）大模型生成耗时较长。决策树生成约 40 秒，主要受外部模型服务影响。后续可以评估本地部署的较小模型，或对排查决策树任务进行微调，在保证质量的前提下缩短响应时间。

（6）系统权限控制尚不完善。当前登录和角色为原型实现，图谱编辑接口没有服务端鉴权。投入实际使用前，需要接入企业统一身份认证，并在后端校验操作权限。

参考文献

[1] 陶家琦, 李心雨, 郑湃, 等. 制造领域知识图谱的应用研究现状与前沿[J]. 计算机集成制造系统, 2022, 28(12): 3720-3736.

[2] HUANG L, YU W, MA W, et al. A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions[J]. ACM Transactions on Information Systems, 2025, 43(2): 1-55.

[3] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems 33. 2020: 9459-9474.

[4] PAN S, LUO L, WANG Y, et al. Unifying large language models and knowledge graphs: a roadmap[J]. IEEE Transactions on Knowledge and Data Engineering, 2024, 36(7): 3580-3599.

[5] 刘晶, 唐震, 王晓茜, 等. 基于元学习的设备故障知识图谱构建及推理方法[J]. 计算机集成制造系统, 2023, 29(11): 3600-3613.

[6] 周彬, 花豹, 陆玉前, 等. 面向设备点检故障根因分析的因果知识建模方法[J]. 计算机集成制造系统, 2023, 29(8): 2708-2721.

[7] 陈宏, 陈新财, 巩晓赟, 等. 基于知识图谱的风电机组诊断系统构建与应用[J]. 郑州大学学报(工学版), 2023, 44(6): 54-60, 98.

[8] 邢雪琪, 丁雨童, 夏唐斌, 等. 基于知识图谱的商用飞机维修方案推荐系统集成建模[J]. 浙江大学学报(工学版), 2023, 57(3): 512-521.

[9] 田嘉鹏, 宋辉, 陈立帆, 等. 面向知识图谱构建的设备故障文本实体识别方法[J]. 电网技术, 2022, 46(10): 3913-3922.

[10] 党小超, 叶汉鑫, 董晓辉, 等. 矿井提升系统的故障实体关系抽取研究[J]. 计算机工程与应用, 2024, 60(16): 311-318.

[11] 夏江镧, 李艳玲, 葛凤培. 基于大语言模型的实体关系抽取综述[J]. 计算机科学与探索, 2025, 19(7): 1681-1698.

[12] OKAZAKI S, KAMINISHI K, FUJIU T, et al. Fault cause identification across

manufacturing lines through ontology-guided and process-aware FMEA graph learning with LLMs[EB/OL]. (2026-08-04)[2026-11-11]. https://arxiv.org/abs/2510.15428v2.

[13] KARPUKHIN V, OGUZ B, MIN S, et al. Dense passage retrieval for open-domain question answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). Online: Association for Computational Linguistics, 2020: 6769-6781.

[14] CHEN J, XIAO S, ZHANG P, et al. M3-Embedding: multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation[C]//Findings of the Association for Computational Linguistics: ACL 2024. Bangkok: Association for Computational Linguistics, 2024: 2318-2335.

[15] GAO Y, XIONG Y, GAO X, et al. Retrieval-augmented generation for large language models: a survey[EB/OL]. (2024-03-27)[2026-11-11]. https://arxiv.org/abs/2312.10997v5.

[16] 刘雪颖, 云静, 李博, 等. 基于大型语言模型的检索增强生成综述[J]. 计算机工程与应用, 2025, 61(13): 1-25.

[17] CORMACK G V, CLARKE C L A, BUETTCHER S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods[C]//Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2009: 758-759.

[18] EDGE D, TRINH H, CHENG N, et al. From local to global: a graph RAG approach to query-focused summarization[EB/OL]. (2025-02-19)[2026-11-11]. https://arxiv.org/abs/2404.16130v2.

[19] PENG B, ZHU Y, LIU Y, et al. Graph retrieval-augmented generation: a survey[J]. ACM Transactions on Information Systems, 2026, 44(2): 1-52.

[20] 冯拓宇, 李伟平, 郭庆浪, 等. 大语言模型增强的知识图谱问答研究进展综述[J]. 计算机科学与探索, 2024, 18(11): 2887-2900.

[21] 张鹤译, 王鑫, 韩立帆, 等. 大语言模型融合知识图谱的问答系统研究[J]. 计算机科学与探索, 2023, 17(10): 2377-2388.

[22] 王永, 秦嘉俊, 黄有锐, 等. 融合知识图谱和大模型的高校科研管理问答系统设计[J]. 计算机科学与探索, 2025, 19(1): 107-117.

[23] 程玉杰, 胡峥, 高永梅, 等. 知识图谱与大模型融合驱动的航空电子装备故障诊断[J]. 航空学报, 2026, 47(19): 233284.

[24] 邓养吾, 刘梦琪, 李荣生, 等. 大模型指令微调驱动的电力故障抢修方案智能生成[J]. 计算机集成制造系统, 2026, 32(9): 3280-3290.

[25] BAHR L, WEHNER C, WEWERKA J, et al. Knowledge graph enhanced retrieval-augmented generation for failure mode and effects analysis[J]. Journal of Industrial Information Integration, 2025, 45: 100807.

[26] OJIMA Y, SAKAJI H, NAKAMURA T, et al. Knowledge management for automobile failure analysis using graph RAG[C]//2024 IEEE International Conference on Big Data (BigData). Washington, DC: IEEE, 2024: 6624-6631.

[27] FRANCIS N, GREEN A, GUAGLIARDO P, et al. Cypher: an evolving query language for property graphs[C]//Proceedings of the 2018 International Conference on Management of Data. Houston: ACM, 2018: 1433-1445.

[28] MALKOV Y A, YASHUNIN D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020, 42(4): 824-836.

[29] XIAO S, LIU Z, ZHANG P, et al. C-Pack: packed resources for general Chinese embeddings[C]//Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. Washington, DC: ACM, 2024: 641-649.

[30] YANG A, LI A, YANG B, et al. Qwen3 technical report[EB/OL]. (2025-05-14)[2026-11-11]. https://arxiv.org/abs/2505.09388.

致谢

【待写：致谢（另起一页，小四宋体，1.35 倍行距，首行缩进两字符）】

附录 A 六元组抽检判定标准

每条记录对照原始故障现象和解决对策，对抽取出的四个字段分别判定“对”或“错”：

（1）部件：发生故障的具体部件。动作词（如换刀、移动）、维修过程（如拆钣金）不是部件；写成设备整体也算错。

（2）故障现象：原始记录中描述的故障表现，可以概括，但意思要一致。

（3）故障原因：能从对策中推断出的原因。原因与现象写成同一句话，或推断不出却硬写的，算错。原始记录确实看不出原因（如只写了“检查中”“交接”）而抽成“未知”的，算对；明明能看出原因却写“未知”的，算错。

（4）处理措施：最终有效的处理动作。只写了中间尝试（如“检查”）而漏掉真正处理的，算错。

拿不准时判“错”，并在备注中写明原因。四个字段都判“对”，该条记录才算整条正确。

附录 B 排查决策树人工评分标准

每条问句下有 5 个决策树，顺序随机，不显示由哪种方法生成。评分者站在现场维修工程师的角度，给每个决策树打 1～5 分：

5 分：原因和处理都对得上这个故障，具体可执行，排查顺序合理，可以直接照着做。

4 分：基本可用，有小问题（某个分支不太相关、顺序可以更好、个别措施不够具体）。

3 分：部分有用，但混有明显无关的原因，或措施笼统，需要工程师自己再判断。

2 分：大部分与该故障不符，或只有空泛的步骤（如“检查设备”“按实际情况处理”）。

1 分：完全不相关，或有误导、可能造成危险的建议，或内容为空。

只看决策树本身是否有用，不猜测由哪种方法生成。大模型辅助评分使用同一标准（温度 0）。